

TECHNICAL REPORT

IRIT/RT-2016-10-FR

Semantics Preservation of the Translation from SIGNAL into Multi-threaded Code

Shuanglong Kan, Jean-Paul Bodeveix, Mamoun Filali

ÉQUIPE ACADIE
{skan, bodeveix, filali}@irit.fr

July 2016

UPS-IRIT, 118 route de Narbonne, 31062 Toulouse CEDEX 9

+33 (0) 561 55 67 65 info@irit.fr www.irit.fr

Abstract

In this report, we introduce a translation from SIGNAL into multi-threaded Java code and prove the semantics preservation of the translation based on Event-B. Intermediate representations: Synchronous Clock Guarded Action (S-CGA), Guarded Action (GA), and Guarded Action with CLusters (GACL) are used to divide the translation into the following 4 steps: (1) translate a SIGNAL program into an S-CGA, (2) translate the S-CGA into a GA, (3) translate the GA into a GACL, and (4) translate the GACL into multi-threaded Java code. Event-B is used to prove the semantics preservation of the translation from GA into GACL with its refinement framework. There are three levels for the refinement. Firstly, GA is specified by an Event-B model (denoted as *mGAbs*) with only a big step, which performs the computation of a reaction in an atomic step. Secondly, the model *mGAbs* is refined by decomposing the big step into a collection of small steps such that each small step corresponds to the computation of a single guarded action (the refined model denoted as *mGA*). Both *mGAbs* and *mGA* are models for GA. The model *mGA* is further refined by giving a partition to GA and each subset in the partition is called a cluster. The refined model is for GACL, denoted as *mGACL*. All proof obligations for these three levels of even-B models are proved to ensure the preservation of safety properties and deadlock freedom preservation.

Keywords

Synchronous languages, SIGNAL, synchronous clock guarded actions, Guarded Action, Guarded Action with Clusters, Event-B, refinement, multi-thread Java code

Contents

Abstract & keywords	1
1 Introduction	5
2 Background	7
2.1 The Modeling Language SIGNAL	7
2.2 Symbolic Labeled Transition System.	8
2.3 Synchronous Clock Guarded Action	8
2.4 Guarded Action	9
2.5 Guarded Action with Clusters	10
3 Framework	11
4 The Translation from SIGNAL into Guarded Actions	14
4.1 Simplified SCGA	14
4.2 The translation from sSCGA into GA	15
4.3 Graph Representation of GA	18
5 From GA to GACL	
Refinement-based Verification	20
5.1 GA Syntax	20
5.1.1 The context <i>GASyntax</i>	20
5.1.2 The context <i>cGA</i>	22
5.2 GA big step semantics	23
5.3 GA small step semantics	24
5.4 Introduction of Clusters into GA	26
5.4.1 Specification of Cluster Construction	27
5.4.2 The machine <i>mGACL</i>	28
5.5 Deadlock Freedom	30
6 Multi-threaded Java Code Generation	34
A GASyntax	39
B cGA	40
C mGAbs	41
D mGA	42
E cGACL	44

F	mGACL	45
G	The Class Diagram of Generated Java Code	46

Chapter 1

Introduction

Safety-critical embedded systems distinguish themselves from general purpose computing systems by several characteristics, such as failure to meet deadlines may cause a catastrophic or at least highly undesirable system failure. Synchronous languages [1], like ESTEREL [2], LUSTRE [3], and SIGNAL [4] are created for designing safety-critical embedded control systems, such as flight control systems in flight-by-wire avionics and antiskidding or anticollision equipment on automobiles. The automatic and safe code generation [5] from specifications is one of the attractive advantages offered by the synchronous development environments. While they share common concerns such as code efficiency and compactness, synchronous languages have developed proper and very different compilation techniques that yield to different code structures. In this report, we focus on the multi-threaded code generation from synchronous language SIGNAL. In addition, we prove the semantics preservation of the code generation using Event-B language, which is a formal method for modeling and verifying the consistency of chains of model refinements. Each step in the translation from SIGNAL into multi-thread code corresponds to a refinement step in Event-B models [6].

SIGNAL as a Modeling Language Safety-critical embedded systems are also considered as reactive systems, because they always interact with their environment continuously. These systems receive from the environment input events, and compute the output. The synchronous approach is an important choice for the design of these systems, which relies on the synchronous hypothesis [1]: a synchronous program reacts to its environment in a sequence of discrete instants. At each instant, the system does input-computation/communication- output, which takes zero time. Even if the physical time is abstracted, the inherent functional properties are not changed, so we can say that this method focuses on functional behaviors at a platform-independent level. In contrast to asynchronous concurrency, synchronous languages avoid the introduction of non-determinism by interleaving. Namely, the execution of two independent, atomic parallel tasks is simultaneous. There are several synchronous languages, such as ESTEREL [2], LUSTRE [3] and QUARTZ [7] based on the perfect synchrony paradigm, and SIGNAL [4] based on the polychrony paradigm. As a main difference from other synchronous languages, SIGNAL naturally considers a mathematical time model, in term of a partial order relation, to describe multi-clocked systems without the necessity of a global clock. This feature permits the description of globally asynchronous locally synchronous systems (GALS) [8] conveniently, where components based on different clock domains are integrated at the system level.

Translation from SIGNAL into multi-threaded Code An integration infrastructure for different synchronous languages has gained a lot of interests in recent years [9, 10]. A classical solution is to use a common intermediate representation. In this report, we introduce the Synchronous

Clock Guarded Action (S-CGA) as the intermediate representation for synchronous languages. So the first step of the multi-threaded code generation from SIGNAL is the translation from SIGNAL into S-CGA. Code generation from S-CGA needs to construct two important data structures: clock hierarchies and Data Dependence Graphs (DDG) [11]. These two structures express the dependence relations between guarded actions in S-CGAs, where clock hierarchies express the clock dependence relations and DDGs express the data dependence relations. Dependence relations indicate the execution order of guarded actions, that is, a guarded action cannot be executed before the guarded actions that it depends on.

Multi-threaded code generation needs to give a legal partition to the guarded actions in an S-CGA. Each subset in the partition is called a cluster, which corresponds to a thread. All guarded actions in a cluster should be executed without being blocked.

Event-B for the Proof of Semantics Preservation. For a safety-critical system, it is required that the compiler must be verified to ensure that the source program semantics is preserved. Event-B is a formal modeling language, which is based on first-order logic and set theory. It can be used to verify the consistency of chains of model refinements. In Event-B, an abstract model can be refined to a concrete model by adding more information, and each model consists of contexts and machines, where contexts define static parts and machines define dynamic parts. The translation from SIGNAL into multi-threaded code proceeds with several steps and each translation step can be viewed as a refinement from a more abstract model. For each refinement, there is a collection of Proof Obligations (PO) that must be proved to ensure the properties, such as invariants and deadlock freedom, of the abstract model are preserved in the concrete model. In this report, we use event-B to model the syntax and semantics of representations of the translation, and prove the semantics preservation of the translation by proving all POs in the Event-B model.

Chapter 2

Background

In this section, we introduce the concepts of SIGNAL language and the intermediate representations used by the translation from SIGNAL into multi-threaded code.

2.1 The Modeling Language SIGNAL

Signals. As declared in the synchronous hypothesis, the behaviors of a reactive system are divided into a sequence of discrete instants. At each instant, the system does input-computation /communication-output, which takes zero time. So, the inputs and outputs are sequences of values, each value of the sequence being present at some instants. Such a sequence is called a *signal*. Consequently, at each instant, a signal may be present or absent (denoted by $\perp$).

	1	2	3	4
$input_1$	1	$\perp$	3	$\perp$...
$input_2$	5	7	9	$\perp$...
$output$	$\perp$	$\perp$	4	$\perp$...

Figure 2.1: Three signals named $input_1$, $input_2$, and $output$

Example 1 Fig. 2.1 illustrates two input signals $input_1$, $input_2$ and one output signal $output$ and their values at instants 1, 2, 3, 4.

Abstract Clock. The set of instants where a signal takes a value is the abstract clock of the signal. Two signals are synchronous if they are always present and absent at the same instants, which means they have the same abstract clock.

In the example given above, the abstract clock of $input_1$, $input_2$ and $output$, denoted respectively $\widehat{input_1}$, $\widehat{input_2}$, and $\widehat{output}$, are defined by different sets of logical instants. For example, $\widehat{input_2} = \{1, 2, 3\}$.

Primitive Constructs of SIGNAL. SIGNAL uses several primitive constructs to express the relations between signals, including relations between values and relations between abstract clocks. Moreover, the primitive constructs can be classified into two families: mono-clock operators (for which all signals involved have the same abstract clock) and multi-clock operators (for which the signals involved may have different clocks).

- Monoclock operators, including *instantaneous function* and *delay*. The instantaneous function $x := f(x_1, \dots, x_n)$ applied on a set of inputs $x_1, \dots, x_n$ will produce the output x , while the delay operator $x := x_1 \$ \text{init } c$ sends a previous value of the input to the output with an initial value c .
- Multiclock operators, including *undersampling* and *deterministic merging*. The undersampling operator $x := x_1$ when x_2 is used to check the output of an input at the true occurrences of another input, while the deterministic merging operator $x := x_1 \text{ default } x_2$ is used to select between two inputs to be sent as the output, with a higher priority to the first input.

SIGNAL also provides some extended constructs to express control-related properties by specifying clock relations explicitly. Since the semantics of extended constructs can be defined in term of the primitive constructs, we will not describe them here and our work focuses on primitive constructs.

2.2 Symbolic Labeled Transition System.

We use Symbolic Labeled Transition Systems (SLTS) [12] to explain the execution of a SIGNAL program. An SLTS is built over a set of variables S of domain D and a set of persistent memories M (conventionally $m_x \in M$ is the memory associated to $x \in S$). Note that a variable has a status while a memory is by essence always present. A valuation $V : S \rightarrow D \cup \{\perp\}$ represents a reaction of the system. A state is a valuation of memories $E : M \rightarrow D$.

SLTSs can be used as the operational semantics of SIGNAL. Consider the following SIGNAL program from [12]:

```
py := y $ 1 init 0 | y := N default py - 1 | N ^= when py <= 0
```

Its corresponding SLTS is shown in Fig. 2.2. It has only two states with respect to m_y . This SLTS is a symbolic representation as each state is labeled by a predicate, which contains more than one explicit states.

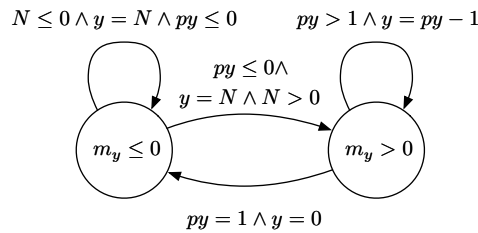


Figure 2.2: An SLTS of the example

2.3 Synchronous Clock Guarded Action

In papers such as [9], clocked guarded actions have been defined as a common representation for synchronous (via synchronous guarded actions), polychronous and asynchronous (via asynchronous guarded actions) models. It has a multi-clocked feature. However, in contrast to the SIGNAL language, clocked guarded actions can evaluate a variable even if its clock does not hold [9] for supporting the asynchronous view. Since we focus on the polychronous view, we introduce S-CGA, which is a variant of clocked guarded actions. S-CGA constrains variable accesses as done by SIGNAL.

Definition 1 An S-CGA system is represented by a set of guarded actions of the form $\gamma \Rightarrow \mathcal{A}$ defined over a set of variables X . The Boolean condition γ is called the guard and $\mathcal{A}$ is called the action. Guarded actions can be of one the following forms:

- (1) $\gamma \Rightarrow x = \tau$ (immediate)
- (2) $\gamma \Rightarrow \text{next}(x) = \tau$ (delayed)
- (3) $\gamma \Rightarrow \text{assume}(\sigma)$ (assumption)

where

- the guard γ is a Boolean condition over the variables of X , their respective clocks (for a variable $x \in X$, we denote its clock $\hat{x}$ and their respective initial clocks (denoted $\text{init}(\hat{x})$),
- τ is an expression over X ,
- σ is a Boolean expression over the variables of X and their clocks.

An immediate assignment $x = \tau$ writes the value of τ immediately to the variable x . The form (1) implicitly imposes that if γ is defined and its value is true, then x is present and τ is defined. Moreover, $\text{init}(\hat{x})$ exactly holds the first instant when x is present. A delayed assignment $\text{next}(x) = \tau$ evaluates τ in the given instant but changes the value of the variable x at next time clock $\hat{x}$ ticks. The form (3) defines a constraint. It determines a Boolean condition which has to hold when γ is defined and true. All the execution traces must satisfy this constraint. Otherwise, they are ignored.

2.4 Guarded Action

Guarded Action (GA) is an intermediate data structure translated from S-CGA by eliminating clocks and special symbols like $\text{next}(x)$, $\text{assume}(\gamma)$, and $\text{init}(\hat{x})$. As programming languages, like C and Java, do not have the notions of clocks and these special symbols, so we need to explain them with variables for code generation.

Definition 2 A GA system is represented by a set of guarded actions of the form $\gamma \Rightarrow x = \tau$ defined over a set of variables X . The guard γ is a Boolean expression over the variables of X . The value of the expression τ is assigned to the variable x when γ is true.

There are two kinds of variables in X of GA: *state variables* and *parameter variables*. State variables are always available during the computation of all reactions, while *parameter variables* are only available during a reaction. In other words, state variables are persistent memories. Parameter variables are temporal memories or registers. State variables are for the *delay* operator in SIGNAL. Even though all clock variables are eliminated from S-CGA, each guarded action in GA also has its own clock, which defines the set of instants that the guarded action should be computed.

There are two kinds of dependence relations between guarded actions: clock dependence and data dependence. For two guarded actions ga_1 and ga_2 , if the clock of ga_1 is a subset of the clock of ga_2 then ga_1 is clock dependent on ga_2 . If the computation of ga_1 needs to read the variable computed in ga_2 then ga_1 is data dependent on ga_2 . Dependence relations are crucial to the partition of GA, which is for multi-threaded code generation.

2.5 Guarded Action with Clusters

The translation from GA into multi-threaded code needs to give a partition for the set of guarded actions in a GA. Each subset in the partition corresponds to a cluster and each cluster corresponds to a thread. The formal definition of partitions is the following:

Definition 3 *A partition Π of a GA is a mapping from the set $\mathcal{G}$ of guarded actions to a set C of clusters in Π , i.e., $\Pi(ga)$ denotes the cluster of a guarded action ga . Let $gas(cl)$ denote the set of guarded actions occurring in the cluster cl , we have $\bigcup_{cl \in C} gas(cl) = \mathcal{G}$ and $gas(cl_1) \cap gas(cl_2) = \emptyset$ for any two clusters cl_1 and cl_2 in C .*

GACL is short for Guard Action with CLusters. The partition methods of GA are based on the dependence relations between guarded actions. There are various criterion to partition guarded actions in GA. In this report, our partition criterion ensures that (1) a cluster can not be blocked during the execution of a reaction, when the first guarded action is activated to be executed, and (2) the execution of all guarded actions in a cluster is sequential.

Chapter 3

Framework

In this chapter, we present the framework of the translation from SIGNAL programs into multi-threaded java code. The translation consists of 4 basic steps: **(1)** translate a SIGNAL program into a S-CGA, **(2)** translate the S-CGA into a GA, **(3)** partition the GA into clusters to obtain a GACL. Each cluster corresponds to a thread in java code. **(4)** translate the GACL into multi-threaded Java code.

S-CGA, GA, and GACL are intermediate representations. For the translation from GA into GACL, we prove the semantics preservation using Event-B. Fig. 3.1 illustrates the framework of multi-threaded code generation from SIGNAL programs.

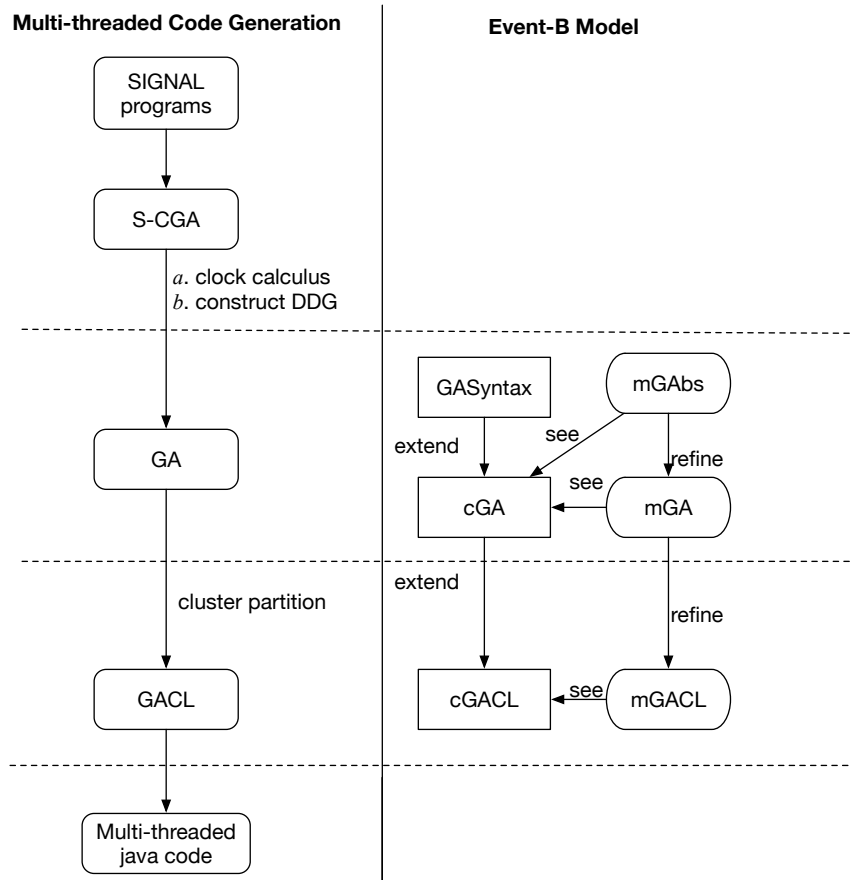


Figure 3.1: The framework of multi-threaded code generation

The translation from an S-CGA into a GA needs to perform *clock calculus* and construct *Data Dependence Graph* (DDG) for the S-CGA. Clock calculus yields a clock hierarchy for the S-CGA. The clock hierarchy and DDG of the S-CGA indicate the execution order of guarded actions in the S-CGA. This translation eliminates clock variables and special symbols. In Event-B part, there are two contexts: *Gasyntax* and *cSCGA* related to S-CGA.

GA is modeled in Event-B with two contexts: *Gasyntax* and *cGA*, and two machines: *mGAbs* and *mGA*. The context *Gasyntax* defines the syntax of general guarded actions. The context *cGA* extends *Gasyntax* and defines constants and axioms used in GA. The machine *mGAbs* models the execution of a GA with only one event: *big step*. A big step is the execution of a reaction in SIGNAL programs. The machine *mGA* refines *mGAbs* by decomposing the big step into several small steps. Each small step executes a guarded action in GA.

The translation from GA into GACL needs to give a partition of GA. Each cluster in the partition corresponds to a thread in multi-threaded code. GACL is modeled in Event-B with a context *cGACL* and a machine *mGACL*. The context *cGACL* extends *cGA* and defines sets, constants, and axioms to represent the partition. The machine *mGACL* refines the *mGA* and describes the behavior of GACL. At last, we translate the GACL into multi-threaded java code by translating each cluster into a thread. All the variables in GACL are directly translated into the variables in java code.

The proof for semantics preservation of the translation from SIGNAL into multi-threaded code is to prove all Proof Obligations (POs) in the Event-B model.

Running Example We present a running example to illustrate our method in the sequel. The running example is a SIGNAL program shown in Fig. 3.2. It contains the four primitive operators in SIGNAL: *Instantaneous relations*, *Delay*, *Undersampling*, and *Deterministic Merging*. One of its execution traces is shown in Fig. 3.3, where *t* denotes *true*, *f* denotes *false*, and $\perp$ denotes the absence of a signal.

```

1: Process Count =
2: (?integer y1;
3:   boolean x1, x2;
4:   !integer y, z;
5: )
6: (| x1 ^ x2
7:   | x1 ^ y2
8:   | y2 := (y1+1) $ init 2
9:   | s1 := y1 when x1
10:  | s2 := y2 when x2
11:  | x := s1 default s2
12:  | y := x + 1
13: )
14: where integer x, y2, s1, s2;
15: end

```

Figure 3.2: A Running example of SIGNAL program

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	...	...
$x1$	t	f	t	t	f	f	t	...	...
$x2$	t	t	f	f	t	f	t	...	...
$y1$	1	3	5	2	4	1	6	...	...
$y2$	2	2	4	6	3	5	2	...	...
$s1$	1	$\perp$	5	2	$\perp$	$\perp$	6	...	...
$s2$	2	2	$\perp$	$\perp$	3	$\perp$	2	...	...
x	1	2	5	2	3	$\perp$	6	...	...
y	2	3	6	3	4	$\perp$	7	...	...

Figure 3.3: A possible execution sequence of the running example

Chapter 4

The Translation from SIGNAL into Guarded Actions

We use an intermediate representation called Synchronous Clock Guarded Action to decompose the complexity of the translation from SIGNAL into GA. More precisely, we first translate SIGNAL programs into SCGAs and then translate SCGAs into GAs.

Our previous work [13] introduced a verified translation from SIGNAL into SCGA. Here we just give an example to illustrate the translation. Consider the running example shown in Fig.??, its corresponding SCGA is shown in Fig. 4.1 (a). Fig. 4.1 (b) shows the correspondence relations between the SIGNAL program and the SCGA. For example, the statement of line 11 in Fig. 3.2 is translated into the statements of line (10), (11), and (12) in Fig. 4.1 (a). The symbol $\hat{s}1$ denotes the clock of $s1$. Line (10) in Fig. 4.1 (a) means that when x is present, the signals $s1$ or $s2$ must be present. Line (11) in Fig. 4.1 (a) means that when $s1$ is present then the assignment $x = s1$ is executed. Line (12) in Fig. 4.1 (a) means that when $s2$ is present and $s1$ is absent then the assignment $x = s2$ is executed.

(1) $true \Rightarrow assume(\hat{x}1 = \hat{x}2)$ (2) $true \Rightarrow assume(\hat{x}1 = \hat{y}2)$ (3) $true \Rightarrow assume(\hat{y}1 = \hat{y}2)$ (4) $init(\hat{y}2) \Rightarrow y2 = 2$ (5) $\hat{y}2 \Rightarrow next(y2) = y1 + 1$ (6) $\hat{s}1 \Rightarrow assume(\hat{y}1 \wedge x1)$ (7) $\hat{y}1 \wedge x1 \Rightarrow s1 = y1$ (8) $\hat{s}2 \Rightarrow assume(\hat{y}2 \wedge x2)$ (9) $\hat{y}2 \wedge x2 \Rightarrow s2 = y2$ (10) $\hat{x} \Rightarrow assume(\hat{s}1 \vee \hat{s}2)$ (11) $\hat{s}1 \Rightarrow x = s1$ (12) $\hat{s}2 \wedge (\neg \hat{s}1) \Rightarrow x = s2$ (13) $\hat{x} \Rightarrow assume(\hat{y})$ (14) $\hat{y} \Rightarrow y = x + 1$	<table> <thead> <tr> <th>SIGNAL</th><th>SCGA</th></tr> </thead> <tbody> <tr> <td>8</td><td>(4)(5)</td></tr> <tr> <td>9</td><td>(6)(7)</td></tr> <tr> <td>10</td><td>(8)(9)</td></tr> <tr> <td>11</td><td>(10)(11)(12)</td></tr> <tr> <td>12</td><td>(13)(14)</td></tr> <tr> <td>6</td><td>(1)</td></tr> <tr> <td>7</td><td>(2)</td></tr> </tbody> </table>	SIGNAL	SCGA	8	(4)(5)	9	(6)(7)	10	(8)(9)	11	(10)(11)(12)	12	(13)(14)	6	(1)	7	(2)
SIGNAL	SCGA																
8	(4)(5)																
9	(6)(7)																
10	(8)(9)																
11	(10)(11)(12)																
12	(13)(14)																
6	(1)																
7	(2)																
(a)	(b)																

Figure 4.1: The SCGA translated from the SIGNAL program in Fig. 3.2

4.1 Simplified SCGA

In this section, we introduce a simplified SCGA, denoted as sSCGA, which has equivalent semantics of SCGA. The simplified SCGA gives a simple presentation of SCGA and it is also helpful to generate efficient code.

Definition 4 A simplified SCGA is represented by a set of guarded actions defined over a set of

variables X , Each guarded action can be one of the following forms:

- (1) $x = \tau$
- (2) $init(x) = c$
- (3) $next(x) = y$
- (4) $\gamma \Rightarrow x = y$
- (5) $\gamma \Rightarrow x = \perp$

where

- x, y is a variable in X and τ is an expression over X ,
- the guard γ is a Boolean condition restricted to the following 4 forms: $\hat{z}, \neg\hat{z}, z, \neg z$, where z is a variable in X ,
- $init(x)$ is the initial value of x and c is a constant,
- $next(x)$ is the next value of x , i.e., the value of x in the next reaction.

The guards in sSCGA have a simpler form, that is, $\hat{z}, \neg\hat{z}, z, \neg z$, while the guards in SCGA can be an expression over variables and their corresponding clocks. SIGNAL programs can be translated into sSCGA according to the rules shown in Table 4.1. Fig. 4.2 illustrates the sSCGA of the running example.

Table 4.1: The translation rules from SIGNAL into sSCGA

SIGNAL	sSCGA
$x = \tau$	$x = \tau$
$x = w \$ init\ c$	$t = w$ (t is a variable) $init(x) = c$ $next(x) = t$
$x = w\ when\ c$	$t = w$ (t is a variable) $c \Rightarrow x = t$ $\neg c \Rightarrow x = \perp$ $\neg\hat{c} \Rightarrow x = \perp$
$x = x1\ default\ x2$	$\hat{x1} \Rightarrow x = x1$ $\neg\hat{x1} \Rightarrow x = x2$

4.2 The translation from sSCGA into GA

The objective of the translation from sSCGA into GA is to eliminate clock variables like $\hat{s1}$ and explain special symbols, like $next(y2)$, using auxiliary variables. We also introduce some optimization for GA.

In order to eliminate clock variables and do some optimization in GA, clock calculus must be performed firstly. Clock calculus is one major analysis of the compilation of SIGNAL programs and it is crucial for determining whether the program is executable and code generation. Clock calculus consists of several steps [11], such as the generation of an clock equation system of relations over clocks; the resolution of the system of clock equations; the construction of a clock

(1) $init(y2) = 2$	(9) $\neg \widehat{x2} \rightarrow \perp$
(2) $next(y2) = y3$	(10) $\widehat{s1} \rightarrow x = s1$
(3) $y3 = y1 + 1$	(11) $\neg \widehat{s1} \rightarrow x = s2$
(4) $x1 \rightarrow s1 = y1$	(12) $y = x + 1$
(5) $\neg x1 \rightarrow \perp$	(13) $(x1 = x1) = (x2 = x2)$
(6) $\neg \widehat{x1} \rightarrow \perp$	(14) $(x1 = x1) = (y2 = y2)$
(7) $x2 \rightarrow s2 = y2$	
(8) $\neg x2 \rightarrow \perp$	

Figure 4.2: The sSCGA of the running example

Table 4.2: The translation rules from sSCGA into clock equations

sSCGA	clock equation
$x = \tau$	$\widehat{x} = \widehat{\tau}$
$init(x)$	no equations
$next(x) = y$	$\widehat{x} = \widehat{y}$
$\gamma \Rightarrow x = y$	$\gamma \rightarrow \widehat{x} = \widehat{y}$
$\gamma \Rightarrow x = \perp$	$\gamma \rightarrow \neg \widehat{x}$

hierarchy. Table 4.2 illustrates the clock equation generation rules of sSCGA. The clock hierarchy of the running example is shown 4.3.

Clock hierarchies can be used to simplify the guards of sSCGA. It consists of the following two steps:

Step 1. Use master clock equivalence class and non-clock variables to define other clock equivalence class. The new definition of each equivalence class is called *clock expression* in the sequel.

Step 2. Replace the master clock with *TRUE*. For each *clock guarded action* in SCGA, we use a clock expression to replace its guard so that to eliminate clock variables.

We now illustrate the above two step by the running example. The running example have 4 equivalence class C_0, C_1, C_2, C_3 and C_0 is the master clock equivalence class. The equivalence classes C_1, C_2, C_3 can be defined as follows:

$$\begin{aligned}
 C_1 &= C_0 \wedge x1 \\
 C_2 &= C_0 \wedge x2 \\
 C_3 &= C_0 \wedge \neg x1 \wedge x2
 \end{aligned}$$

After replacing C_0 with *TRUE*, we can obtain that $C_1 = x2$, $C_2 = x2$, and $C_3 = \neg x1 \wedge x2$. The simplified SCGA of the running example by replacing guards with clock expressions is shown in the left part of Fig. 4.4.

The symbols $init(y2)$ and $next(y2)$ in Fig. 4.4 (a) need to be further explained. We first consider the semantics of $next(y2)$. The symbol $next(y2)$ denotes the value of $y2$ at the next reaction. Therefore the value of $next(y2)$ will not be used in the computation of the current reaction and only be used in the next reaction. In order to translate sSCGA into multi-threaded code, the semantics of $next$ needs to be explained using auxiliary variables. We use the auxiliary

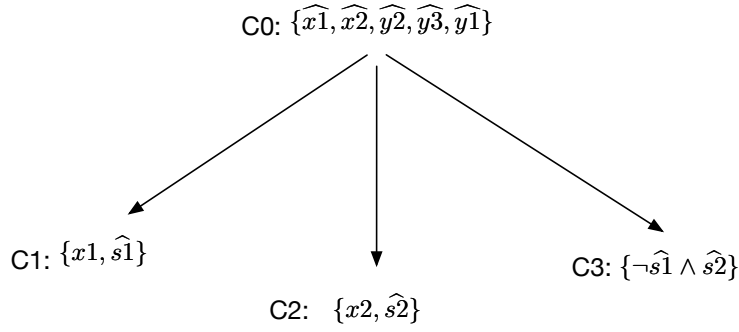


Figure 4.3: The clock hierarchy of the running example

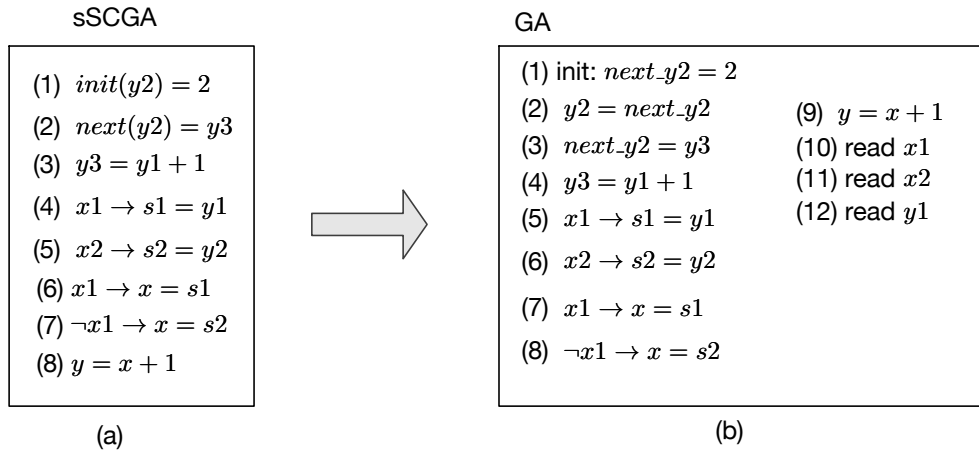


Figure 4.4: The translation from sSCGA into GA

variable $next_y2$ to denote the value of $y2$ at the next reaction. The variable $next_y2$ is also called the *next variable* of $y2$ in the sequel.

In Fig. 4.4 (a), we can observe that, the relation between $next(y2)$ and $y2$ is implicit. That is, no guarded actions have both $next(y2)$ and $y2$. With the introduction of $next_y2$, we add a new guarded action $y2 = next_y2$. This guarded action has the same guard as the guarded action that writes $y2$. (The guard of $next_y2 = y3$ is $TRUE$). It must be computed before (1) the execution of all guarded actions that read $y2$ and (2) the guarded action that writes $next_y2$. The reason why $y2 = next_y2$ should be computed before $next_y2 = y3$ is that $next_y2$ already has the value for $y2$ at current reaction. The computation of $next_y2$ is for remembering the value of $y2$ at the next reaction.

We now translate the symbol $init(y2) = 2$, which means that initially, the value of $y2$ is 2. This initialization is done before any reaction. It is translated as $init : next_y2 = 2$, where $init$ means that it is an initialization. The reason why we initialize $next_y2$ instead of $y2$ is that in each reaction, we first set $y2 = next_y2$ when $y2$ is present. We will further explain it in Section ??.

The GA translated from the simplified SCGA of the running example is shown in Fig. 4.4 (b). In the GA, three new guarded actions (10) (11), and (12) denote that the values of $x1$, $x2$, and $x3$ are read from the environment. So they are non-deterministic.

4.3 Graph Representation of GA

In this subsection, we are concerned with building a graph representation of GA. We begin with building a Data Dependence Graph (DDG) of a GA and then some simplification rules are introduced for the graph.

1. Data Dependence Graph. Data dependence graphs indicate the execution orders of guarded actions. We first define the reading and writing data dependencies. Let $\gamma \rightarrow x = \tau$ denotes a guarded action, where γ is the guard and $x = \tau$ is the action.

Definition 5 Let $FV(\tau)$ denote the free variables occurring in the expression τ . The reading and writing dependencies from guarded actions to variables are defined as follows, respectively:

$$\begin{aligned} RdVars(\gamma \rightarrow x = \tau) &:= FV(\gamma) \cup FV(\tau) \\ WrVars(\gamma \rightarrow x = \tau) &:= \{x\}. \end{aligned}$$

The reading and writing dependencies from variables to guarded actions are defined as follows

$$\begin{aligned} RdActs(x) &:= \{\gamma \rightarrow \mathcal{A} \mid x \in RdVars(\gamma \rightarrow \mathcal{A})\} \\ WrActs(x) &:= \{\gamma \rightarrow \mathcal{A} \mid x \in WrVars(\gamma \rightarrow \mathcal{A})\}. \end{aligned}$$

The relations $RdVars$ and $RdActs$ are reading dependencies, and $WrVars$ and $WrActs$ are writing dependencies. We now define the data dependence graph of GA.

Definition 6 The data dependence graph of a GA is a directed graph $\langle \mathcal{V}, \mathcal{E} \rangle$, where $\mathcal{V}$ is a set of vertexes representing guarded actions and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is a set of edges representing the dependencies between guarded actions. For two vertexes identified with the two guarded actions ga_1 and ga_2 , which are not initialization of variables, there is an edge from ga_1 to ga_2 iff one of the following two conditions holds:

- (1) there exists a variable x , which has a next variable $next_x$, such that $x \in WrVars(ga_2)$ and $next_x \in WrVars(ga_1)$,
- (2) there exists a variable x , which has no next variable, such that $x \in WrVars(ga_2) \cap RdVars(ga_1)$.

An edge from ga_1 to ga_2 means that ga_1 is dependent on ga_2 . In other words, ga_1 must be executed after ga_2 . Fig. 4.5 illustrates the DDG of the running example. We need to pay more attention to the guarded actions (a) $y2 = next_y2$ and (b) $next_2 = y3$. We can observe that the guarded action (b) should be executed after (a). But the guarded action (a) needs the variable $next_y2$, which means that $next_y2$ is always available when $y2$ is present. We will further explain the differences between next variables and variables in Section ??.

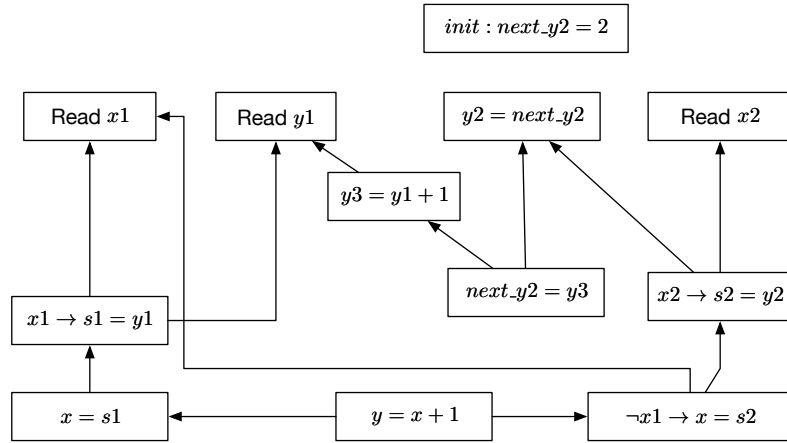


Figure 4.5: The DDG of the running example

Chapter 5

From GA to GACL

Refinement-based Verification

In this section, we concentrate on the translation from GA into GACL and the proof of semantics preservation using Event-B. From Section 5.1 to 5.3, we build the event-B model of GA with two levels: *big step semantics level* and its refinement *small step semantics level*. In Section 5.4, we introduce the translation from GA into GACL and build the event-B model of GACL. The semantics preservation of the translation can be proved by proving all POs in the event-B models.

An Event-B model consists of two parts: contexts and machines. Contexts contain the static parts of a model, namely, carrier sets, constants, axioms, and theorems, whereas machines contain the dynamic parts of a model, namely, variables, invariants, theorems, variants, and events.

5.1 GA Syntax

The Event-B model of GA includes two contexts: *Gasyntax* and *cGA* (see Appendix A and B). The context *Gasyntax* defines the syntax and semantics of guards and actions. The context *cGA* extends *Gasyntax* to define the semantics of guarded actions and dependence relations between guarded actions.

5.1.1 The context *Gasyntax*

As shown in Definition 2. A guarded action can be represented as $\gamma \Rightarrow x = \tau$, where γ is a guard, x is a variable, and τ is an expression. We use three sets in *Gasyntax* to represent these three notions: *Guard* is a set of guards, *Variable* is a set of variables, and *Expression* is a set of expressions over variables and values. The constants in *Gasyntax* are responsible for defining the semantics of GA. Both sets and constants in *Gasyntax* are the following:

sets Guard Expression Variable
constants nextVar State semG semE semA Value init Action lhs rhs Input inp_dom

We now explain each constants in details. (1) *Value*: the domain of all variables (defined by the axiom *Value_def*). It is the set $\mathbb{N}$ of all natural numbers. Note that, for boolean variables, we use 0 to denote *FALSE* and 1 to denote *TRUE* in the model. (2) *State*: all possible partial maps from variables to values (defined by the axiom *State_def*). It is a crucial notion in the model to define the semantics of expressions and guards.

@action Action = Variable $\times$ Expression

(3) *Action*: the set of assignments from expressions to variables. In fact, it is the right part of a guarded action with respect to $\Rightarrow$. An action is denoted as (v, e) , where v is a variable and e is an expression. (4) *lhs*: the *left hand side* of an action (defined by the axiom *lhs*). (5) *rhs*: the *right hand side* of an action (defined by the axiom *rhs*).

```
@semE_ty semE ∈ Expression × State → P(Value)
@semE_prop ∀ e, st, st'. e ∈ Expression ∧ st ∈ State ∧ st' ∈ State ∧ st ⊆ st' ⇒
semE(e ↦ st) ⊆ semE(e ↦ st')
```

(6) *semE*: the semantics of expressions. It maps an expression e and a state s to a subset V of *Value*. V is the set of all possible values of e evaluated in the state s . For most expressions, the set V has only one element. But for non-deterministic inputs, V may have more than one element. Therefore we use a set to denote the evaluation of an expression. Note that, an expression must be evaluated in a *state* by replacing the variables in the expression with the corresponding values in the *state*. The axiom *semE_prop* indicates that *semE* is a non-decreasing function with respect to states.

```
@semA_ty semA ∈ Action × State → P(State)
@semA_def semA = (λ (t ↦ e) ↦ st. st ∈ State | {v · v ∈ semE(e ↦ st) | st ≺ {t ↦ v}})
```

(7) *semA*: the semantics of actions (defined by the axioms *semA_ty* and *semA_def*). It maps an action and a state to a set of states, which are the all possible states after executing the action from the state. The axiom *semA_def* defines the relations between *semE* and *semA*. For a guarded action $\gamma \Rightarrow v = \tau$, assume the current state is s , then all possible states after executing the action are obtained by binding the value $\text{semE}(v \mapsto s)$ to v .

```
@semG_ty semG ∈ Guard × State → BOOL
```

(8) *semG*: the semantics of guards (defined by the axiom *semG_ty*). It evaluates a guard in a state and returns a Boolean value. (9) *Input*: the set of input variables (defined by the axiom *Input_def*). (10) *inp_dom*: the domain of input variables (defined by the axiom *inp_dom*).

```
@nextVars_ty nextVar ∈ Variable ⇔ Variable // nextVar(x) = nex
@next_prop dom(nextVar) ∩ ran(nextVar) = ∅
@init_ty init ∈ State
```

(11) *init*: the initial state of GA (defined by *init_ty*). The domain of *init* is the set of next variables. It means that initially, all next variables have values. In fact, next variables are *state variables*. (12) *nextVar*: the partial map from variables to their next variables (defined by the axiom *nextVars_ty*). The axiom *next_prop* indicates that no variable is both a state variable and a parameter variable.

```
theorem @semA_mono2 ∀ a, st, st'. st ∈ State ∧ a ∈ Action ∧ st' ∈ semA(a ↦ st) ⇒
dom(st') = dom(st) ∪ {lhs(a)}
theorem @semA_mono3 ∀ a, st, st'. st ∈ State ∧ a ∈ Action ∧ st' ∈ semA(a ↦ st) ⇒
dom(st) ⊆ dom(st')
theorem @semA_mono1 ∀ a, st, st'. a ∈ Action ∧ st ∈ State ∧
st' ∈ State ∧ st' ∈ semA(a ↦ st) ⇒ {lhs(a)} ≺ st ⊆ {lhs(a)} ≺ st'
```

There are three theorems for the constants *semA*: *semA_mono1*, *semA_mono2*, and *semA_mono3*, indicating the relations between the two states before and after an action. They can be proved by using the axioms in *Gasyntax* and they are used for the proof of POs in the Event-B models. Theorem *semA_mono2* indicates that after executing an action, the written variable in the action has a value in the new state. Theorem *semA_mono3* can be easily derived from *semA_mono2*. Theorem *semA_mono1* indicates the subset relation between the current state and the next state after executing an action. But the variable of $\text{lhs}(a)$ is not included. The reason is that when $\text{lhs}(a)$ is a next variable, it may have a value in the current state but have a different value in the next state.

5.1.2 The context *cGA*.

The context *cGA* extends the context *GAyntax*. It specifies the relations between guarded actions in GA and some important properties of guarded actions that are used in the proof of POs.

@GuardAction_ty GuardedAction $\subseteq$ Guard $\times$ Action
 @GA_finite finite (GuardedAction)

There are two new constants *GuardedAction* and *used* in *cGA*. the constant *GuardedAction* corresponds to the set of all guarded actions in a GA. It is defined by the axioms *GuardAction_ty* and *GA_finite*. The axiom *GA_finite* indicates that *GuardedAction* is a finite set.

@guardedActions_prop $\forall g_1, a_1, g_2, a_2, st. g_1 \mapsto a_1 \in \text{GuardedAction} \wedge$
 $g_2 \mapsto a_2 \in \text{GuardedAction} \wedge st \in \text{State} \wedge (g_1 \mapsto a_1) \neq (g_2 \mapsto a_2) \wedge$
 $\text{used}(g_1 \mapsto a_1) \subseteq \text{dom}(st) \wedge \text{used}(g_2 \mapsto a_2) \subseteq \text{dom}(st)$
 $\wedge \text{semG}(g_1 \mapsto (\text{ran}(\text{nextVar}) \triangleleft st)) = \text{TRUE} \wedge$
 $\text{semG}(g_2 \mapsto (\text{ran}(\text{nextVar}) \triangleleft st)) = \text{TRUE} \Rightarrow \text{lhs}(a_1) \neq \text{lhs}(a_2)$

The axiom *guardedActions_prop* specifies that there do not exist a state and two guarded actions such that the two guards are evaluated to be true in the state and the two actions write the same variable. In other words, during the computation of a reaction, no variable can be written more than once.

The constant *used* is defined to specify dependence relations between guarded actions. The constant *used* for a guarded action denotes the set of variables that must be computed before it.

@used_ty used $\in$ GuardedAction $\rightarrow \mathbb{P}(\text{Variable})$

In GA, each guarded action computes the value of a variable. The constant *used* indicates that some variables must be computed after some other variables. We explain the constant *used* through Fig. 4.5. For each guarded action *ga*, *used(ga)* is the set of variables written by the guarded actions that *ga* depends on. In Fig. 4.5, we use arrows to represent the dependence relations. We need to further explain the constant *used* for next variables. In Fig. 4.5, there are guarded actions that compute *y2* and *next_y2*, respectively. The computation of *next_y2* does not need to read the variable *y2*, but we will still add *y2* to *used(next_y2 = y3)*. The reason is that the computation of *y2* must be performed before *next_y2*. Putting *y2* into *used(next_y2 = y3)* enforces this computation order. In fact, *y2* reads the value of *next_y2* computed before this reaction. As *next_y2* is a state variable, so it is always present. The relation between *used* and the arrows (which are dependence relations) in DDGs is the following: let *ga* be a guarded action, and $\{ga_1, \dots, ga_n\}$ be a set of guarded actions that it depends on, i.e., there is an arrow from *ga* to *ga_i* with $1 \leq i \leq n$. We have $\bigcup_{1 \leq i \leq n} \text{WrVars}(ga_i) = \text{used}(ga)$. The axioms *used_property*, *sem_next_used*, *next_prop*, *next_prop2* specify these properties of the constant *used*. The axiom *used_property* specifies that for each variable which does not have a corresponding next variable, its computation does not depend on any next variables. The axiom *next_prop* specifies that for a guarded action $g \mapsto a$, if one of its used variables is in the codomain of *nextVar* then *lhs(a)* is in the domain *nextVar*. The axiom *sem_next_used* specifies that for a guarded action *ga* whose action is of the form $y = \text{next}_y$, *next_y* is in *used(ga)*. As *next_y* is persistent and remembers the value of *y* in previous reaction, so the computation of *y* does not depend on the computation of *next_y* in current reaction. The axiom *next_prop2* specifies that for a guarded action of the form $g \rightarrow x = \text{next}_x$, it has the only one next variable *next_x* in *used(g $\rightarrow$ x = next_x)*.

@used_property $\forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{lhs}(a) \notin \text{dom}(\text{nextVar}) \Rightarrow$
 $\text{used}(g \mapsto a) \in \mathbb{P}(\text{Variable} \setminus \text{ran}(\text{nextVar}))$
 @sem_next_used $\forall g, a. g \mapsto a \in \text{GuardedAction} \Rightarrow$
 $\text{nextVar}[\{\text{lhs}(a)\}] \subseteq \text{used}(g \mapsto a)$

$$\text{@next_prop } \forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \cap \text{ran}(\text{nextVar}) \neq \emptyset \Rightarrow \text{lhs}(a) \in \text{dom}(\text{nextVar})$$

$$\text{@next_prop2 } \forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{lhs}(a) \in \text{dom}(\text{nextVar}) \Rightarrow \text{nextVar}(\text{lhs}(a)) \in \text{used}(g \mapsto a) \wedge (\text{ran}(\text{nextVar}) \setminus \{\text{nextVar}(\text{lhs}(a))\}) \cap \text{used}(g \mapsto a) = \emptyset$$

The axiom *used_property1* specifies that if all used variables of a guarded action have values in a state then the expression of the guarded action always has a value evaluated in the state. The axiom *used_property2* says that for a guarded action and two states, if the used variables of the guarded action have the same values in both states then the expression of the guarded action has the same value evaluated in both states.

$$\begin{aligned} \text{@used_property1 } & \forall g, a, st. st \in \text{State} \wedge g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \subseteq \text{dom}(st) \Rightarrow \text{semE}(\text{rhs}(a) \mapsto st) \neq \emptyset \\ \text{@used_property2 } & \forall g, a, st, st'. st \in \text{State} \wedge st' \in \text{State} \wedge g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \subseteq st = \text{used}(g \mapsto a) \subseteq st' \Rightarrow \text{semE}(\text{rhs}(a) \mapsto st) = \text{semE}(\text{rhs}(a) \mapsto st') \end{aligned}$$

5.2 GA big step semantics

The model of GA contains two machines *mGAbs* and *mGA*. The machine *mGAbs* specifies the computation of a reaction in a single atomic event called *big_step*. The machine *mGA* refines *mGAbs* by decomposing the *big_step* into a sequence of small steps so that they can be executed concurrently.

The data-flow operational semantics of GA can be explained by SLTS [12]. Fig. 5.1 illustrates a part of the SLTS of the running example. The initial state of SLTS is $m_y = 2$. The memory variable m_y is, in fact, the next variable of y in the GA, i.e., *next_y*. We only illustrate 3 states and 3 transitions of the SLTS. Each transition is labeled by a set of vectors and only one vector is shown for each transition in the figure. For example, the transition from $m_y = 2$ to $m_y = 2$ is labeled by a set of vectors and one of them is $[t, t, 1, 2, 1, 2, 1, 2]$. Elements in the vector from left to right correspond to the values of $x1, x2, y1, y2, s1, s2, x, y$, respectively. The symbols t, f , and $\perp$ denote *true*, *false*, and absence, respectively.

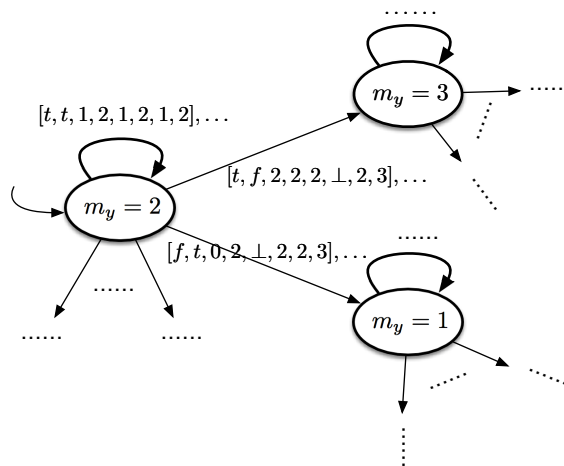


Figure 5.1: The SLTS of the running example

The code of the machine *mGAbs* is shown in Appendix C. It has a variable *state*, which corresponds to the current state of an SLTS. Therefore *state* is responsible for storing the values

of next variables. It is initialized to be *init* in the event *INITIALISATION*. The constant *init* contains the initial values of all next variables (see the axiom *init_dom* in Appendix A).

The event *big_step* corresponds to the computation of a reaction. The code of *big_step* is the following:

```

event big_step
  any state1
  where
    @state_1 state1 ∈ State
    @state_2 state1 ≠ ∅

    @state_4 ∀ g, v, e · g ↦ (v ↦ e) ∈ GuardedAction ∧
      (g ↦ (v ↦ e)) ↦ state1 ∈ enabled ∧ v ∉ dom(nextVar) ⇒ v ∈ dom(state1) ∧
        state1(v) ∈ semE(e ↦ state1)
    @state_5 ∀ x · x ∈ dom(nextVar) ∩ dom(state1) ⇒ state1(x) = state(nextVar(x))
  then
    @state_re state := (ran(nextVar) ⋈ state) ⋈ state1
end

```

It has a parameter *state1*, which stores the values of variables after the computation of a reaction. In fact, it is one of the vectors labeled on transitions of SLTSs. There are four guards for *big_step* under the keyword **where**. The actions of the *big_step* are executed only when all these guards are evaluated to be true. The meanings of the guards *state_1* and *state_2* are intuitive. As explained in Section 5.1, for a variable *y*, which has a unique next variable *next_y*. If *y* is present in a reaction then the value of *y* in the reaction is the value of *next_y* computed in the previous reaction. The guard *state_4* is for the variables which have no next variable while the guard *state_5* is for the variables which have a unique next variable. The guard *state_4* indicates that for any guarded action, if it is enabled then the variable *v*, which has no next variable, computed in the guarded action, has a value in *state1* and the value is in *semE*(*e* ↦ *state1*). The guard *state_5* indicates that for all variables *x*, if $x \in \text{dom}(\text{nextVar})$ then the value of *x* is the value of *next_x* computed in previous reaction. The only action in *big_step* modifies the variable *state* with the new values of next variables in *state1*.

5.3 GA small step semantics

The machine *mGA* refines *mGAbs* by decomposing the *big_step* into a sequence of small steps. Let $\mathcal{G} = \{ga_1, \dots, ga_n\}$ be a GA translated from a SIGNAL program, where ga_i with $1 \leq i \leq n$ is a guarded action. The event *big_step* in *mGAbs* executes all guarded actions, that are enabled in a reaction, in an atomic step. The refinement of *big_step* is shown in Appendix D. The code illustrates that the *big_step* in *mGAbs* is decomposed into *n* small steps and a *big_step* in *mGA*. Each small step refines an implicit event of Event-B *skip* and executes a guarded action in $\mathcal{G}$. The execution trace of a reaction in *mGA* is a sequence of *small_steps* ended with a *big_step*. That is, a sequence like: $\langle \text{small_step}, \dots, \text{small_step}, \text{big_step} \rangle$. Note that, *small_steps* in a reaction can execute concurrently. Therefore, for each reaction, we may have more than one execution traces. More formally, The set of traces (both finite and infinite) in *mGAbs* can be expressed by the regular expression

$$\text{INITIALIZATION } ((\text{small_step})^+ \text{ big_step})^*$$

The refinement includes: (1) introducing two new variables: *done* and *r_state*, (2) adding a new event *small_step*, (3) refining the event *big_step*, and (4) adding a collection of invariants. Some invariants specify the relations between the variables in *mGAbs* and *mGA*. The variable *done* denotes the set of guarded actions that have been executed in a reaction. The variable *r_state* corresponds to the combination of *state* and *state1* in the machine *mGAbs*. It stores the values of variables that have been computed in a reaction and the values of all state variables.

a. *small_step* The new event *small_step* executes a single guarded action when the guarded action is enabled. The code of *small_step* is the following:

```

convergent event small_step
  any g a
  where
    @g_ty g  $\mapsto$  a  $\in$  GuardedAction \ done
    @cl_ready used(g  $\mapsto$  a)  $\subseteq$  dom(r_state)
    @guard_ok semG(g  $\mapsto$  (ran(nextVar)  $\triangleleft$  r_state)) = TRUE
  then
    @a_cl r_state  $\Leftarrow$  semA(a  $\mapsto$  r_state)
    @d_cl done: = done  $\cup$  {g  $\mapsto$  a}
end

```

There are three conditions in the guard of the event (in *where* part), which specify when the guarded action is enabled.

- the condition *g_ty* indicates that the guarded action should not have been executed before, i.e., in *GuardedAction \ done*.
- all the variables that are read in the computation of the guarded action should have values in *r_state*, i.e., they have been computed.
- the guard of the guarded action should be evaluated to be true in *r_state*.

If all these conditions in the guard are true then the actions of the event are executed. The action part includes (1) first update *r_state* by adding a new binding of variable to value and (2) then add the guarded action to *done*, which means that it has been executed.

b. *big_step* The event *big_step* refines the event *big_step* in *mGAbs*. The code of *big_step* is the following:

```

event big_step refines big_step

  where
    @r_state_not_empty r_state  $\neq \emptyset$ 
    @finished  $\forall g, a \cdot$ 
      (g  $\mapsto$  a  $\in$  GuardedAction \ done  $\wedge$ 
       used(g  $\mapsto$  a)  $\subseteq$  dom(r_state))
       $\Rightarrow$  semG(g  $\mapsto$  (ran(nextVar)  $\triangleleft$  r_state)) = FALSE

  with
    @state1 state1 = r_state

  then
    @pc done: =  $\emptyset$ 
    @rst r_state: = ran(nextVar)  $\triangleleft$  r_state
end

```

Its guard has two conditions. The first condition is intuitive and inherited from the *big_step* in *mGAbs*. The second condition says that guarded actions, which are not executed, are also disabled. In other words, all guarded actions that are enabled have been executed. More precisely, if a guarded action is not executed and all its used variables have values in *r_state* then the guard of the guarded action is *FALSE*. The actions of *big_step* initialize the next reaction: (1) reset the variable *done* and (2) modify the values of next variables. The witness of the event (The part with the keyword **with**) indicates the relations between *r_state* and *state1* in *mGAbs*.

c. Invariants in *mGA*. The invariants of *mGA* should be preserved before and after all events. In fact, invariants are safety properties preserved in the refined models.

```

@r_state_ty r_state ∈ State
@done_ty done ⊆ GuardedAction
@done_used ∀ g, a. g ↦ a ∈ done ⇒ used( g ↦ a ) ⊆ dom(r_state)
@dom_done_r_state dom(r_state) \ ran(nextVar) = prj1[prj2[done]] \ ran(nextVar)
@done_semG ∀ g, a. g ↦ a ∈ done ⇒
    semG(g ↦ ran(nextVar) ≤ r_state) = TRUE
@done_lhs ∀ g, a. g ↦ a ∈ done ⇒ lhs(a) ∈ dom(r_state)
@done_r_state ∀ g, a. g ↦ a ∈ done ∧ lhs(a) ∉ dom(nextVar) ⇒
    r_state(lhs(a)) ∈ semE(rhs(a) ↦ r_state)
@init_invariant ran(nextVar) ⊆ dom(r_state)
@lhs_invariant ∀ g, a. g ↦ a ∈ GuardedAction \ done ∧ used(g ↦ a) ⊆ dom(r_state)
    ∧ semG(g ↦ ran(nextVar) ≤ r_state) = TRUE ⇒
    lhs(a) ∉ dom(r_state) \ ran(nextVar)

```

We explain each invariants in the following: The invariants r_state_ty , $done_ty$ are intuitive. The invariants $dom_done_r_state$, $done_used$, $done_lhs$, and $done_r_state$ specify the relations between $done$ and r_state . The invariant $done_used$ indicates that for each guarded action, if it has been executed then its used variables have values in r_state . The invariant $dom_done_r_state$ means that all variables except next variables that have values in r_state are also the written variables of the guarded actions in $done$ except next variables. The invariant $done_semG$ means that the guards of all guarded actions that have been executed are *TRUE*. The invariant $done_lhs$ indicates that if a guarded action has been executed then the variable which it writes has a value in r_state . The invariant $done_r_state$ means that for each action (v, e) , the value of v is the evaluation of the expression e in the current state. The invariant $init_invariant$ indicates that all next variables always have values in r_state , since next variables are memories. The invariant $lhs_invariant$ means that every variable in a GA can only be written once in a reaction. More precisely, for each guarded action which has not been executed, if it is enabled in the current state then its written variable is in $dom(r_state) \setminus ran(nextVar)$. The variant $GuardedAction \setminus done$ indicates that the number of small steps in a reaction is finite.

5.4 Introduction of Clusters into GA

The translation from a GA into a GACL is to give a partition of the set of guarded actions in the GA. Each subset of guarded actions, which is called *cluster*, in the partition corresponds to a thread. Fig. 5.2 shows the GACL of the running example. There are nine clusters, which correspond to the nine threads in the figure. In the Event-B model, a cluster is modeled as a sequence of guarded actions shown in Fig. 5.3, which has the following properties:

- (1). each guarded action in the cluster has an index, ranging from 1 to the size of the cluster. A guarded action with the index i is called the i th guarded action,
- (2). the indexes indicate the execution order of the guarded actions. That is, all guarded actions are executed from 1 to n in sequence.,
- (3). when the first guarded action of the cluster is executed then the execution of the cluster can not be *blocked*,
- (4). the execution of the i th guarded action depends on the execution of the $(i - 1)$ th guarded action. More precisely, the execution of the i th guarded action needs the variable computed in the $(i - 1)$ th guarded action.

The term *blocked* for a guarded action in property (3) means that there are some variables read in the guarded action have not been computed. Properties (3) and (4) indicate that when the $(i - 1)$ th guarded action has been executed, then all used variables of the i th guarded action have been computed.

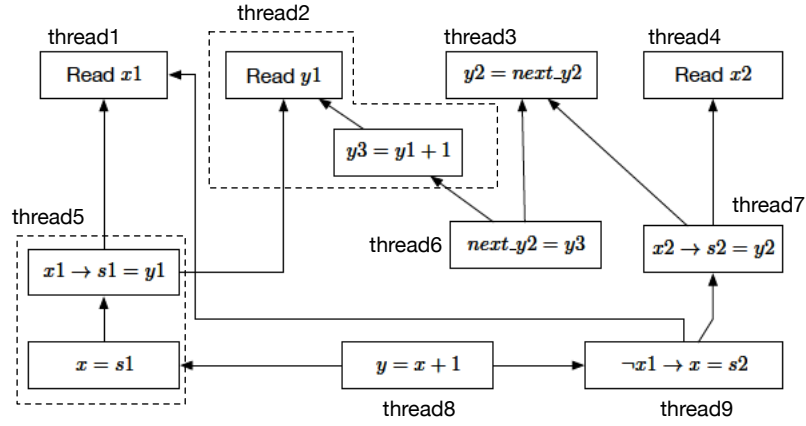


Figure 5.2: The GACL of the running example

1	$g_1 \rightarrow a_1$
2	$g_2 \rightarrow a_2$
...	...
n	$g_n \rightarrow a_n$

Figure 5.3: The illustration of a cluster

We give a partition method for GA such that each cluster in a partition conforms to the 4 properties. The principle of our partition method can be described as follows:

Rule.1 Initially, each guarded action in the GA is a cluster in the partition. That is, each cluster contains only one unique guarded action.

Rule.2 For two clusters c_1 and c_2 , if they satisfy that c_1 is the only father of c_2 then merge c_1 and c_2 into one cluster.

Initially, each guarded action is a cluster, so clusters satisfy the 4 properties. Two clusters can merge into one cluster only when the first guarded action of one cluster only depends on the last guarded action of the other one. Therefore the merged new cluster satisfies the 4 properties. The GACL in Fig. 5.2 uses this partition method.

5.4.1 Specification of Cluster Construction

The Event-B model of *GACL* consists of a context *cGACL* and a machine *mGACL*. The difference between the model of GA and the model of *GACL* is the introduction of a partition.

We first introduce the context *cGACL*, which extends the context *cGA*. The context *cGACL* adds a set *Cluster* and 3 constants:

sets	<i>Cluster</i>
constants	<i>guardedAction</i> <i>clusterSize</i> <i>cl_used</i>

Each element in *Cluster* corresponds to a cluster in a partition. The constant *guardedAction* defines the relation between clusters and guarded actions with indexes.

$@\text{guardedAction_ty}; \text{guardedAction} \in \text{Cluster} \rightarrow (\mathbb{N} \rightarrow \text{GuardedAction})$ $@\text{guardedAction_dom} \forall \text{cl} \cdot (\text{cl} \in \text{Cluster} \Rightarrow \text{dom}(\text{guardedAction}(\text{cl})) = 1.. \text{clusterSize}(\text{cl}))$ $@\text{GuardAction_constrain} \text{GuardedAction} = (\text{UNION } \text{cl} \cdot$

$$\begin{array}{l} cl \in \text{Cluster} \mid \text{guardedAction}(cl)[1..clusterSize(cl)] \\ @cl_used_ty \quad cl_used \in \text{Cluster} \rightarrow \mathbb{P}(\text{Variable}) \\ @cl_used_def \quad cl_used = (\lambda cl \cdot (cl \in \text{Cluster}) \mid used(\text{guardedAction}(cl)(1))) \end{array}$$

The axioms *guardedaction_ty* and *guardedAction_dom* formally define the type and the domain of *guardedAction*. The domain of *guardedAction*(*cl*) for a cluster *cl* is the range from 1 to the number of the guarded actions in *cl*. The constant *clusterSize* defines the number of guarded actions in a cluster (see axiom *clusterSize*). The axiom *guardedAction_constrain* specifies the relations between *GuardedAction* and clusters. The constant *GuardedAction* is the union of all guarded actions in clusters. The constant *cl_used* (see axiom *cl_used_ty*) defines the set of variables used by a cluster. The concept of *cl_used* is similar to the constant *used*. The constant *cl_used* maps a cluster to a set of variables that the cluster depends on. If all these variables have values then all guarded actions in the cluster can be executed without being blocked. In fact, *cl_used* is the set of used variables of the first guarded action in the cluster (see axiom *cl_used_def*).

Other axioms in *cGACL* link the sets and constants in *cGACL* and *cGA* and specify the 4 properties of clusters.

$$@disjointClusters \quad \forall cl1, cl2 \cdot \text{ran}(\text{guardedAction}(cl1)) \cap \text{ran}(\text{guardedAction}(cl2)) = \emptyset \Rightarrow cl1 = cl2$$

1). Axioms *GuardedAction_constrain* and *disjointClusters* specify that the set *Cluster* is a partition of guarded actions (*GuardedAction* is defined in *cGA*). That is, the union of all clusters is the set of all guarded action and the intersection of any two clusters is empty.

$$\begin{array}{l} @size_spec \quad \forall cl, i \cdot i \in 1..clusterSize(cl) \Rightarrow \\ \quad used(\text{guardedAction}(cl)(i)) \subseteq used(\text{guardedAction}(cl)(1)) \cup \\ \quad (\{ g, a \cdot g \mapsto a \in \text{guardedAction}(cl)[1..i-1] \mid lhs(a) \}) \\ @cluster_next \quad \forall cl, i \cdot i \in 2..clusterSize(cl) \Rightarrow lhs(\text{prj}_2(\text{guardedAction}(cl)(i))) \end{array}$$

$$\begin{array}{l} @used_cl_ga \quad \forall cl, i, j \cdot i \in \text{dom}(\text{guardedAction}(cl)) \wedge j \in \text{dom}(\text{guardedAction}(cl)) \wedge \\ \quad lhs(\text{prj}_2(\text{guardedAction}(cl)(j))) \in used(\text{guardedAction}(cl)(i)) \wedge \\ \quad lhs(\text{prj}_2(\text{guardedAction}(cl)(i))) \notin \text{dom}(\text{nextVar}) \Rightarrow (i > j) \end{array}$$

2). Axioms *cluster_next*, *used_cl_ga* and *size_spec* specify the property (4) that the *i*th guarded action depends on the (*i* − 1)th guarded action. 3). Axiom *no_two_father* specifies the property (3) that a guarded action in a cluster, except the first guarded action, is not depend on guarded actions in other clusters.

$$\begin{array}{l} @no_two_father2 \quad \forall cl, i \cdot i \in 2..clusterSize(cl) \\ \Rightarrow (\forall ga1, ga2 \cdot ga1 \in \text{GuardedAction} \wedge ga2 \in \text{GuardedAction} \wedge \\ \quad lhs(\text{prj}_2(ga1)) = lhs(\text{prj}_2(\text{guardedAction}(cl)(i-1))) \wedge \\ \quad lhs(\text{prj}_2(ga1)) = lhs(\text{prj}_2(ga2)) \Rightarrow ga1 = ga2) \end{array}$$

4). Theorem *after_next* specifies that the computation of next variables can only appear at the end of a cluster.

$$\begin{array}{l} \text{theorem } @after_next \quad \forall cl, i \cdot i \in 2..clusterSize(cl) \Rightarrow \\ \quad lhs(\text{prj}_2(\text{guardedAction}(cl)(i-1))) \notin \text{ran}(\text{nextVar}) \end{array}$$

5.4.2 The machine mGACL

The machine *mGACL* refines the machine *mGA* by considering the information of the partition. In fact, *mGACL* does not add any new event and only refines the guards and actions of events in *mGA* with the information of the partition.

A new variable *pc* is added to *mGACL* to represent the program counters of clusters. For example, *pc*(*cl*) denotes the program counter of the cluster *cl*. But the variable *done* in *mGA* is

removed in *mGACL*. It can be represented by the variable *pc* and clusters. The variable *r_state* is the same as in *mGA*. The invariants *pc_ty* and *pc_ok* define the variable *pc*, including its type, domain and codomain.

```
@pc_ty pc ∈ Cluster → ℕ
@pc_ok ∀cl · cl ∈ Cluster ⇒ pc(cl) ∈ 1 .. clusterSize(cl) + 1
```

The invariant *done* links the variable *done* in *mGA* and the constant *Cluster* and the variable *pc* in *mGACL*. It means that all guarded actions in a cluster before *pc(cl)* have been computed. In other words, they are in *done*.

```
@done done = ( cl · cl ∈ Cluster | guardedAction(cl)[1..pc(cl)-1])
```

The axiom *r_state_invariant* indicates that the variable computed by a guarded action in *done* has a value in *r_state*. The invariant *used_notin_r_state* indicates that for each guarded action with the index greater than *pc(cl)*, there exist used variables of the guarded action, which have no values in *r_state*.

```
@r_state_invariant ∀cl · union(used[guardedAction(cl)[1..pc(cl)-1]) ⊆ dom(r_state)
@used_notin_r_state ∀cl, i · i > pc(cl) ∧ i ∈ dom(guardedAction(cl)) ⇒
    used(guardedAction(cl)(i)) ⊄ dom(r_state)
```

The event *INITIALIZATION* in *mGACL* initializes the variables *r_state* and *pc*. The variable *done* in *mGA* can be represented by the set *Cluster* and the variable *pc* as the invariant *done* indicates.

a. *small_step*. The event *small_step* has a new parameter *cl* which indicates a cluster. Using *cl* we can express the guard of *small_step* with clusters. The difference between the guards of *small_steps* in *mGA* and *mGACL* is the checking of used variables. In *mGA*, for each guarded action, we need to check whether the used variables of a guarded action have values in *r_state*. In *mGACL*, we only need to check whether the used variables of the first guarded action in the cluster have values in *r_state*, since the partition rule ensures that all used variables of *pc(cl)*th guarded action always have values in *r_state*. The action *small_step* modifies *r_state* to record the effect of the execution of the guarded action, and *pc(cl)* to execute the next guarded action in *cl*.

```
event small_step refines small_step
any cl g a
where
    @cl_ty cl ∈ Cluster
    @cl_ready cl_used(cl) ⊆ dom(r_state)
    @guard_act_dom pc(cl) ∈ dom(guardedAction(cl))
    @guard_act_val (g ↦ a) = guardedAction(cl)(pc(cl))
    @guard_ok semG(g ↦ ran(nextVar) ≤ r_state) = TRUE
then
    @a_cl r_state ⊔ semA(a ↦ r_state)
    @pc_next pc(cl) := pc(cl) + 1
end
```

b. *big_step*. The event *big_step* in *mGACL* rewrites the guard of *big_step* in *mGA* using clusters. In fact they are equivalent to each other. The guard expresses that a reaction finishes if all guarded actions that have not been executed are blocked or their guards are false. The invariant *used_notin_r_state* has a relation with the guard. The action of *big_step* initializes the variables *r_state* and *pc* for the next reaction.

```
event big_step refines big_step
where
    @r_state_not_empty r_state ≠ ∅
    @finished ∀cl · (cl ∈ Cluster ∧
```

```

pc(cl) ∈ dom(guardedAction(cl)) ∧
cl_used(cl) ⊆ dom(r_state))
⇒ semG(prj1(guardedAction(cl)(pc(cl))) ↦ ran(nextVar) ≤ r_state) = FALSE
then
  @pc pc := Cluster × {1}
  @st r_state := ran(nextVar) < r_state
end

```

5.5 Deadlock Freedom

In this subsection, we consider the deadlock freedom properties in the event-B models of GA and GACL. There are two kinds of deadlock freedom: (1) the deadlock freedom of a single machine. It is responsible for proving that a single machine runs forever, (2) the relative deadlock freedom. It is concerned about the proof of the deadlock freedom *preservation* between an abstract model and its direct refined model (concrete model). More precisely, if the abstract model is deadlock free then the concrete model is also deadlock free. Fig. 5.4 illustrates the proof obligation of the deadlock freedom preservation. It means that if there exists one of the guards in the abstract model is evaluated to be true then there exists one of the guards in the concrete model is evaluated to be true.

$ \begin{array}{c} A(c) \\ I(c, v) \\ J(c, v, w) \\ G_1(c, v) \vee \dots \vee G_m(c, v) \\ \vdash \\ H_1(c, w) \vee \dots \vee H_n(c, w) \end{array} $	c : constants v : the variables in abstract model w : the variables in concrete model $A(c)$: Axioms $I(c, v)$: Abstract invariants $J(c, v, w)$: Concrete invariants $G_1(c, v) \vee \dots \vee G_m(c, v)$: Disjunction of abstract guards $H_1(c, w) \vee \dots \vee H_n(c, w)$: Disjunction of concrete guards
---	---

Figure 5.4: The proof obligations rule of deadlock freedom preservation

Guards in events with parameters. The encoding of guards G_1 to G_m and H_1 to H_n needs to consider parameters in events, since the scopes of parameters in events are local, whereas the deadlock freedom POs are global. We need to use $\exists$ quantifier for parameters. For an event with parameters $x_1, \dots, x_n$ and the guard G , its corresponding formula for guard is

$$\exists x_1, \dots, x_n. G.$$

In the event-B models $mGAbs, mGA, mGACL$, there are 5 events with 5 guards. These guards are the following:

(1) The guard of *big_step* in $mGAbs$ (denoted as G_big_GAbs) is:

```

state1 ∈ State ∧ state1 ≠ ∅ ∧ (∀ g, a · g ↦ a ∈ GuardedAction ∧
used( g ↦ a ) ⊆ dom(state1) ∧ semG(g ↦ ran(nextVar) ≤ state1) = TRUE
∧ lhs(a) ∉ dom(nextVar) ⇒
lhs(a) ∈ dom(state1) ∧ state1(lhs(a)) ∈ semE(rhs(a) ↦ state1)) ∧
(∀ x · x ∈ dom(nextVar) ∩ dom(state1) ⇒ state1(x) = state(nextVar(x)))

```

where the variable *state1* is a parameter. Therefore it can be specified with $\exists$ quantifier:

$$\exists state1. G_big_GAbs.$$

(2) The guard of *small_step* in mGA (denoted as G_small_GA) is:

$$g \mapsto a \in \text{GuardedAction} \setminus \text{done} \wedge \text{used}(g \mapsto a) \subseteq \text{dom}(r_state) \\ \wedge \text{semG}(g \mapsto (\text{ran}(\text{nextVar}) \triangleleft r_state)) = \text{TRUE}$$

where the variables g, a are parameters. Therefore it can be specified with $\exists$ quantifier:

$$\exists g, a. G_small_GA.$$

(3) The guard of *big_step* in *mGA* (denoted as G_big_GA) is:

$$\forall g, a \cdot (g \mapsto a \in \text{GuardedAction} \setminus \text{done} \wedge \\ \text{used}(g \mapsto a) \subseteq \text{dom}(r_state)) \\ \Rightarrow \text{semG}(g \mapsto (\text{ran}(\text{nextVar}) \triangleleft r_state)) = \text{FALSE}$$

This event has no parameters. Therefore it can be specified without $\exists$ quantifier:

$$G_big_GA.$$

(4) The guard of *small_step* in *mGACL* (denoted as G_small_GACL) is:

$$cl \in \text{Cluster} \wedge cl_used(cl) \subseteq \text{dom}(r_state) \wedge \\ pc(cl) \in \text{dom}(\text{guardedAction}(cl)) \wedge \\ (g \mapsto a) = \text{guardedAction}(cl)(pc(cl)) \wedge \\ \text{semG}(g \mapsto \text{ran}(\text{nextVar}) \triangleleft r_state) = \text{TRUE}$$

where the variables cl, g, a are parameters. Therefore it can be specified with $\exists$ quantifier:

$$\exists cl, g, a. G_small_GACL.$$

(5) The guard of *big_step* in *mGACL* (denoted as G_big_GACL) can be specified as:

$$\forall cl \cdot (cl \in \text{Cluster} \wedge \\ pc(cl) \in \text{dom}(\text{guardedAction}(cl)) \wedge \\ cl_used(cl) \subseteq \text{dom}(r_state)) \\ \Rightarrow \text{semG}(\text{prj}_1(\text{guardedAction}(cl)(pc(cl))) \mapsto \text{ran}(\text{nextVar}) \triangleleft r_state) = \text{FALSE}$$

This event has no parameters. Therefore it can be specified without $\exists$ quantifier:

$$G_big_GACL.$$

Deadlock Freedom Preservation for *mGAb*s and *mGA*. In the abstract model *mGAb*s, there is only one event *big_step*. In the concrete model *mGA*, there are two events *small_step* and *big_step*. So the PO of the deadlock freedom preservation can be specified as:

$$(\exists state1. G_big_GAb) \Rightarrow (\exists g, a. G_small_GA \vee G_big_GA).$$

Note that, there is a witness for *big_steps* of these two models: $state1 = r_state$, where $state1$ is a parameter of *big_step* in *mGAb*s and r_state is a variable in *mGA*. But we do not need to be concerned about this witness. In fact, it has no relations with the POs of deadlock freedom.

The proof idea of this dead lock freedom PO is to prove the left part of the PO is permanently true, that is:

$$(\exists g, a. G_small_GA \vee G_big_GA) \equiv \text{TRUE}$$

.

In fact,

$$\neg(\exists g, a. G_small_GA) = \forall g, a. \neg G_small_GA \equiv G_big_GA$$

.

Therefore,

$$(\exists g, a. G_small_GA \vee G_big_GA) \equiv (\neg G_big_GA \vee G_big_GA) \equiv \text{TRUE}.$$

The proof indicates that there always exists an event in *mGA*, which is enabled. So the machine *mGA* runs forever.

Deadlock Freedom Preservation for mGA and $mGACL$. The machine $mGACL$ refines the two events in mGA but does not add any new events. The PO for the deadlock freedom preservation of mGA and $mGACL$ can be specified as follows:

$$(\exists g, a. G_small_GA \vee G_big_GA) \Rightarrow (\exists cl, g, a. G_small_GACL \vee G_big_GACL).$$

In order to simplify the proof of this PO, we can prove a more stronger condition, which consists of two POs.

1. $(\exists g, a. G_small_GA) \Rightarrow (\exists cl, g, a. G_small_GACL),$
2. $G_big_GA \Rightarrow G_big_GACL.$

In fact, the first one specifies that the guard of small step in mGA implies the guard of small step in $mGACL$. The second one specifies that the guard of big step in mGA implies the guard of big step in $mGACL$. The original PO is decomposed into two smaller POs.

The right part of the first PO is $\exists cl, g, a. G_small_GACL$. We first rewrite it to facilitate the description as follows: $\exists cl, g_1, a_1. G_small_GACL(g/g_1, a/a_1)$, where $G_small_GACL(g/g_1, a/a_1)$ is obtained by replacing g with g_1 and a with a_1 in G_small_GACL . and $G_small_GACL(g/g_1, a/a_1)$ can be decomposed into the following 5 statements.

- P1. $cl \in Cluster,$
- P2. $cl_used(cl) \subseteq dom(r_state),$
- P3. $pc(cl) \in dom(guardedAction(cl)),$
- P4. $(g_1 \mapsto a_1) = guardedAction(cl)(pc(cl)),$
- P5. $semG(g_1 \mapsto ran(nextVar) \triangleleft r_state) = TRUE.$

The key to prove this PO is to prove that there exists a cluster cl such that $guardedAction(cl)(pc(cl)) = g \mapsto a$. That is, $g_1 = g$ and $a_1 = a$. According to axiom *GuardAction_constrain* in the context $cGACL$:

$$GuardedAction = (\cup cl. cl \in Cluster \mid guardedAction(cl)[1..clusterSize(cl)]),$$

There must exists a cluster cl such that $g \mapsto a = guardedAction(cl)(x)$. Then we only need to prove $x = pc(cl)$. There are 3 cases to prove it:

- (1). $x < pc(cl)$. As $g \mapsto a \in GuardedAction/done$ and the invariant *done* in $mGACL$ is

$$done = (\cup cl. cl \in Cluster \mid guardedAction(cl)[1..pc(cl) - 1],$$

therefore this case is impossible.

- (2) $x > pc(cl)$. The invariant *used_notin_r_state*:

$$\forall cl, i. i > pc(cl) \wedge i \in dom(guardedAction(cl)) \Rightarrow used(guardedAction(cl)(i) \notin dom(r_state))$$

indicates that $used(guardedAction(cl)(x) \notin dom(r_state)$. But the invariant *cl_ready* in mGA is: $used(g \mapsto a) \subseteq dom(r_state)$. Therefore this case is impossible.

- (3) $x = pc(cl)$. As the cases (1) and (2) are false, this case must be true. Therefore, P1, P3, P4, P5 can be proved.

Now we consider the proof of P2. According to the context $cGACL$, $cl_used(cl)$ is the set of used variables of the first guarded action in the cluster cl . There are two cases:

(1) $pc(cl) \neq 1$. According to the invariant $r_state_invariant$

$$\forall cl. union(used[guardedAction(cl)[1..pc(cl) - 1]]) \subseteq dom(r_state),$$

P2 can be proved easily.

(2) $pc(cl) = 1$. We have $cl_used(cl) = used(g \mapsto a)$. Therefore P2 is proved.

We now prove the second PO: $G_big_GA \Rightarrow G_big_GACL$. The key of this PO is to prove that for all clusters cl , which satisfies $pc(cl) \in dom(guardedAction(cl))$ and $cl_used(cl) \subseteq dom(r_state)$, the guarded action $guardedAction(cl)(pc(cl))$ is $g \mapsto a$, which satisfies $g \mapsto a \in GuardedAction \setminus done$ and $used(g \mapsto a) \subseteq dom(r_state)$. More precisely, we need to prove:

- $guardedAction(cl)(pc(cl)) \in GuardedAction \setminus done$,
- $used(guardedAction(cl)(pc(cl))) \subseteq dom(r_state)$.

The first one can be proved by the invariant $done$ in $mGACL$:

$$done = (\cup cl. cl \in Cluster \mid guardedAction(cl)[1..pc(cl) - 1]).$$

The second one can be proved by the axiom $size_spec$ in the context $cGACL$:

$$\begin{aligned} \forall cl, i. i \in 1..clusterSize(cl) \Rightarrow used(guardedAction(cl)(i)) \subseteq \\ (\{g, a. g \mapsto a \in guardedAction(cl)[1..i - 1] \mid lhs(a)\} \cup \\ used(guardedAction(cl)(1))), \end{aligned}$$

and the invariant $r_state_invariant$ in the machine $mGACL$:

$$\forall cl. union(used[guardedAction(cl)[1..pc(cl) - 1]]) \subseteq dom(r_state).$$

Chapter 6

Multi-threaded Java Code Generation

In this section, we introduce the multi-threaded Java code generation from GACL. Fig. 6.1 illustrates the structure of the generated java code and its relation with GACL. It consists of two parts. The first part corresponds to the context of GACL and the second part corresponds to the machine of GACL. Generated Java code for GACL context implements the notions of guarded actions, clusters, and partitions. Generated Java code for GACL machine implements a thread pool, which consists of a collection of worker threads for executing tasks. These two parts are connected by the relation between clusters and tasks. A task is responsible for executing a cluster and invoking all its next clusters, which are ready to be executed. The class diagram of the generated multi-threaded Java code is shown in Appendix G.

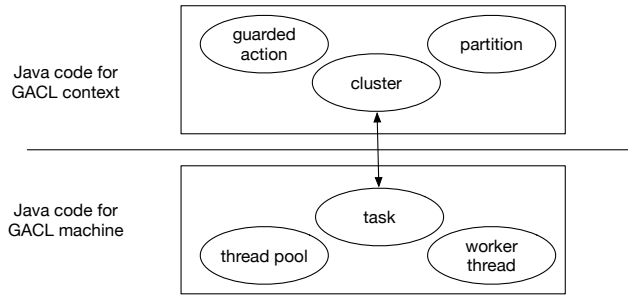


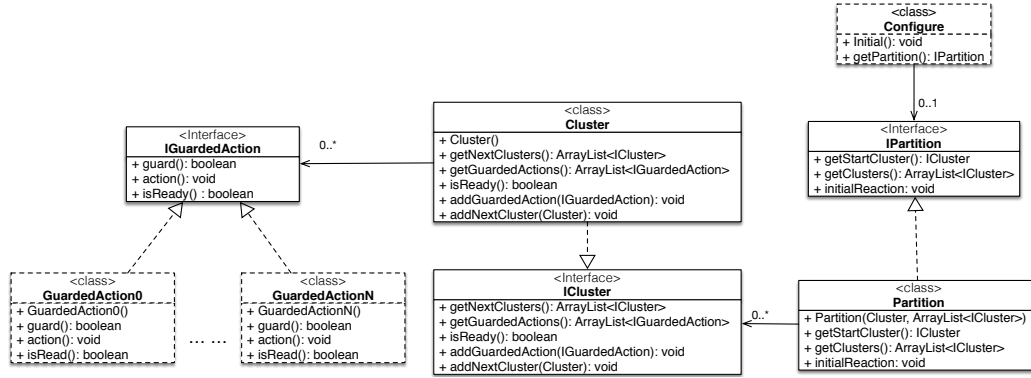
Figure 6.1: The structure of generated java code

Java code for the context *cGACL*. The class diagram of Java code for *cGACL* is shown in Fig. 6.2. This part contains interfaces: *IGuardedAction*, *ICluster*, and *IPartition* and their corresponding implementation classes. Table 6.1 illustrates the correspondence between the class diagram and *cGACL*.

The classes in Fig. 6.2 with dashed lines are classes that should be modified for different SIGNAL programs. But we do not need to modify other classes for any SIGNAL program, i.e., they are fixed. We now introduce the class diagram in details. For the interface *IGuardedAction*, it consists of three methods: (1) *guard()*, which corresponds to the guard of a guarded action. It returns *true* when the guard is evaluated to be true, otherwise, it returns false. (2) *action()*, which corresponds to the action of the guarded action. It executes the action of the guarded action. (3) *isReady()*, which returns whether the guarded action is ready to be executed. In fact, *isReady()* checks whether all used variables (defined by the constant *used* in *cGA*) have values in the computation. For each guarded action in GACL, we create a class implemented the interface *IGuardedAction*. In Fig. 6.2, the classes *GuardedAction0* to *GuardedActionN* represent that there are *N* guarded actions in GACL.

Table 6.1: The correspondence between class diagram and *cGACL*

constants and sets in <i>cGACL</i>	classes and interfaces in class diagram
<i>GuardedAction</i>	The interface <i>IGuardedAction</i> and the classes <i>GuardedAction0</i> , ..., <i>GuardedActionN</i>
<i>Action</i>	The methods <i>action()</i> in <i>IGuardedAction</i> , <i>GuardedAction0</i> , ..., <i>GuardedActionN</i>
<i>Guard</i>	The methods <i>guard()</i> in <i>IGuardedAction</i> , <i>GuardedAction0</i> , ..., <i>GuardedActionN</i>
<i>used</i>	The method <i>isReady()</i> in <i>IGuardedAction</i> , <i>GuardedAction0</i> , ..., <i>GuardedActionN</i>
<i>Cluster</i>	The interface <i>ICluster</i> and the class <i>Cluster</i>
<i>cl_used</i>	The method <i>isReady()</i> in <i>ICluster</i> and <i>Cluster</i>

Figure 6.2: The class diagram of Java code for *cGACL*

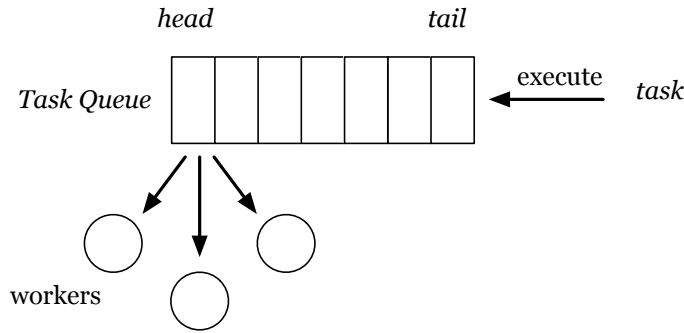
For the interface *ICluster*, it corresponds to a cluster of guarded actions. It has 5 methods: (1) *addGuardedAction*, adds a guarded action to the tail of the cluster. In a cluster, guarded actions are stored in a list. (2) *getGuardedActions*, returns the list of guarded actions in a cluster. (3) *addNextCluster*, builds the relations between different clusters. Each cluster has a list of next clusters. All next clusters must be executed after this cluster. This method adds a cluster to the list of next clusters. (4) *getNextClusters*, returns the list of next clusters. (5) *isReady*, checks whether the cluster is ready to be executed. In fact, *isReady()* checks whether all used variables of the cluster (defined by the constant *cl_used* in *cGACL*) have values in the computation. The class *Cluster* implements the interface *ICluster*.

For the interface *IPartition*, it stores all clusters in GACL. It consists of 3 methods: (1) *initialReaction*, initializes the data used in the computation. (2) *getStartCluster*, returns the list of clusters that are executed at the beginning of a reaction. (3) *getClusters*, returns the list of clusters in a partition.

The class *Configure* builds a partition of GACL. The method *Initial* creates the relations between guards, clusters, and the relations between clusters. The context part stores all information of GACL. In fact, for a new GACL, multi-threaded code generation only needs to modify the classes with the dashed line.

Java code for the machine *mGACL*. This part is responsible for the scheduling of the execution of clusters. Fig. 6.3 illustrates the scheduling of threads. Thread pool consists of a task queue and a set of workers. Each task corresponds to a cluster in a partition. Each worker is a thread, which performs the computations of tasks.

The scheduling of the thread pool can be described as follows: When a task (a task is cluster) is ready to be executed, it is put in the tail of the task queue. When there is a task in the head of the queue, a free worker is allocated for the computation of the task. Then the status of the worker is *busy* and the task is removed from the queue. After performing the computation of the task, the worker is in *free* status and it waits for another task. The class diagram for the machine *mGACL* has interfaces and classes, which correspond to the notions of the task, worker, and thread pool.



ltq: the length of the task queue

wpt: the number of workers that are performing tasks.

invariant: $\text{numofactivetask} = \text{ltq} + \text{wpt}$

when **numofactivetask** becomes zero after the first task being inserted into the queue, the whole reaction terminates.

Figure 6.3: The framework of thread pool

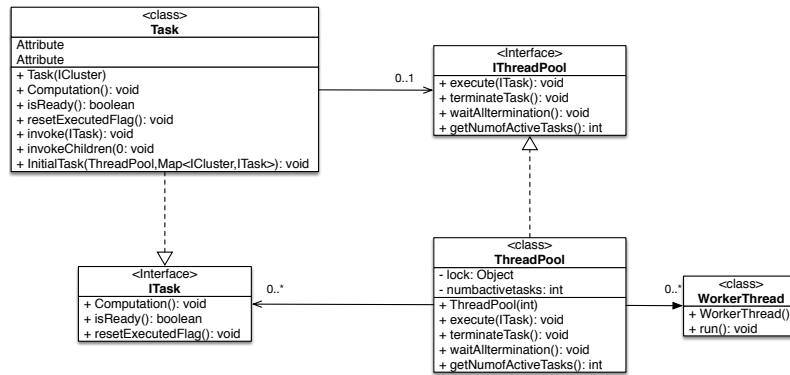


Figure 6.4: The class diagram of Java code for *mGACL*

Fig. 6.4 illustrates the class diagram for *mGACL*. The interface *ITask* defines all operations of a task. It has 3 methods: (1) *Computation*, performs the computation of a task. (2) *isReady*, checks whether the task is ready to be executed, in fact, it is the checking of *isReady* in the class *Cluster*. (3) *resetExecutedFlag*, resets that the computation of a task has not been performed at the beginning of each reaction. The class *Task* implements all these methods. In addition, it also has two methods for the invocation of its next clusters. (4) *invoke*, invokes a task, which

checks whether a next cluster is ready to be executed. If the next cluster is ready then put it in the tail of the task queue. (5) *invokeChildren*, invokes all its next clusters.

The class *WorkerThread* implements the functionality of a worker. It has only one method *run*, which activates the thread. It waits for the queue and performs the computation of tasks.

The interface *IThreadPool* defines methods to implement the scheduling of the thread pool. (1) *execute*, allocates a worker thread for a task in the queue. (2) *terminateTask*, checks whether all tasks, which are enabled in a reaction, have been executed. If all tasks have been executed then it notifies the main thread for the termination of the reaction. (3) *getNumofActiveTasks*, returns the number of tasks in the task queue. The class *ThreadPool* implements all these methods.

Bibliography

- [1] Albert Benveniste, Paul Caspi, Stephen A. Edwards, Nicolas Halbwachs, Paul Le Guernic, and Robert de Simone. The synchronous languages 12 years later. *Proceedings of the IEEE*, 91(1):64–83, 2003.
- [2] Frederic Boussinot and R De Simone. The estereel language. *Proceedings of the IEEE*, 79(9):1293–1304, 1991.
- [3] N Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous data flow programming language lustre. *Proceedings of the IEEE*, 79(9):1305–1320, 1991.
- [4] Albert Benveniste, Paul Le Guernic, and Christian Jacquemot. Synchronous programming with events and relations: the SIGNAL language and its semantics. *Sci. Comput. Program.*, 16(2):103–149, 1991.
- [5] Loic Besnard, Thierry Gautier, and Jeanpierre Talpin. Code generation strategies in the polychrony environment. 2009.
- [6] Aryllo G Russo. Modeling in event-b - system and software engineering by jean-raymond abrial. *ACM Sigsoft Software Engineering Notes*, 36(2):38–39, 2011.
- [7] Paul Le Guernic, Jean pierre Talpin, and Jean christophe Le Lann. Polychrony for system design. *JOURNAL FOR CIRCUITS, SYSTEMS AND COMPUTERS*, 12:261–304, 2002.
- [8] Milos Krstic, Eckhard Grass, Frank K. Gürkaynak, and Pascal Vivet. Globally asynchronous, locally synchronous circuits: Overview and outlook. *IEEE Design & Test of Computers*, 24(5):430–441, 2007.
- [9] Jens Brandt, Mike Gemünde, Klaus Schneider, Sandeep K. Shukla, and Jean-Pierre Talpin. Representation of synchronous, asynchronous, and polychronous components by clocked guarded actions. *Design Autom. for Emb. Sys.*, 18(1-2):63–97, 2014.
- [10] Jean-Pierre Talpin, Jens Brandt, Mike Gemünde, Klaus Schneider, and Sandeep K. Shukla. Constructive polychronous systems. *Sci. Comput. Program.*, 96:377–394, 2014.
- [11] Mirabelle Nebut. An overview of the signal clock calculus. *Electr. Notes Theor. Comput. Sci.*, 88:39–54, 2004.
- [12] M Nebut. *Réactions synchrones: spécification et analyse (French for: Synchronous reactions: specification and analysis)*. PhD thesis, PhD thesis, University of Rennes 1, France, 2002.
- [13] Zhibin Yang, Jean-Paul Bodeveix, Mamoun Filali, Kai Hu, Yongwang Zhao, and Dianfu Ma. Towards a verified compiler prototype for the synchronous language SIGNAL. *Frontiers of Computer Science*, 10(1):37–53, 2016.

Appendix A

Gasyntax

```

context Gasyntax

sets Guard Expression Variable

constants nextVar State semG semE semA Value init Action lhs rhs Input inp_dom

axioms
  @Value_def Value =  $\mathbb{N}$ 
  @Input_def Input  $\subseteq$  Variable
  @inp_dom inp_dom  $\in$  Input  $\rightarrow \mathbb{P}(\text{Value})$ 
  @State_def State = Variable  $\leftrightarrow$  Value
  @semG_ty semG  $\in$  Guard  $\times$  State  $\rightarrow \text{BOOL}$ 
  @semG_mono  $\forall g, st, st'. g \in \text{Guard} \wedge st \in \text{State} \wedge st' \in \text{State} \wedge st \subseteq st' \wedge$ 
    semG(g  $\mapsto$  st) = TRUE  $\Rightarrow$ 
    semG(g  $\mapsto$  st') = TRUE
  @action Action = Variable  $\times$  Expression
  @lhs lhs = ( $\lambda a. a \in \text{Action} \mid \text{prj}_1(a)$ )
  @rhs rhs = ( $\lambda a. a \in \text{Action} \mid \text{prj}_2(a)$ )
  @semE_ty semE  $\in$  Expression  $\times$  State  $\rightarrow \mathbb{P}(\text{Value})$ 
  @semE_prop  $\forall e, st, st'. e \in \text{Expression} \wedge st \in \text{State} \wedge st' \in \text{State} \wedge st \subseteq st' \Rightarrow$ 
    semE(e  $\mapsto$  st)  $\subseteq$  semE(e  $\mapsto$  st')
  @semA_ty semA  $\in$  Action  $\times$  State  $\rightarrow \mathbb{P}(\text{State})$ 
  @semA_def semA = ( $\lambda (t \mapsto e) \mapsto st. st \in \text{State} \mid \{v \cdot v \in \text{semE}(e \mapsto st) \mid st \triangleleft \{t \mapsto v\}\}$ )
  theorem @semA_mono2  $\forall a, st, st'. st \in \text{State} \wedge a \in \text{Action} \wedge st' \in \text{semA}(a \mapsto st) \Rightarrow$ 
    dom(st') = dom(st)  $\cup \{\text{lhs}(a)\}$ 
  theorem @semA_mono3  $\forall a, st, st'. st \in \text{State} \wedge a \in \text{Action} \wedge st' \in \text{semA}(a \mapsto st) \Rightarrow$ 
    dom(st)  $\subseteq$  dom(st')
  theorem @semA_mono1  $\forall a, st, st'. a \in \text{Action} \wedge st \in \text{State} \wedge$ 
    st'  $\in \text{State} \wedge st' \in \text{semA}(a \mapsto st) \Rightarrow \{\text{lhs}(a)\} \triangleleft st \subseteq \{\text{lhs}(a)\} \triangleleft st'$ 
  @nextVars_ty nextVar  $\in$  Variable  $\leftrightarrow$  Variable // nextVar(x) = nex
  @next_prop dom(nextVar)  $\cap$  ran(nextVar) =  $\emptyset$ 
  @init_ty init  $\in$  State
  @init_dom dom(init) = ran(nextVar)
end

```

Listing A.1: Static features of the splitting refinement

Appendix B

cGA

```

context cGA extends GASyntax

constants GuardedAction used enabled

axioms
  @GuardAction_ty GuardedAction  $\subseteq$  Guard  $\times$  Action
  @GA_finite finite (GuardedAction)
  @used_ty used  $\in$  GuardedAction  $\rightarrow \mathbb{P}(\text{Variable})$ 
  @used_property  $\forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{lhs}(a) \notin \text{dom}(\text{nextVar}) \Rightarrow$ 
    used( $g \mapsto a$ )  $\in \mathbb{P}(\text{Variable} \setminus \text{ran}(\text{nextVar}))$ 
  @sem_next_used  $\forall g, a. g \mapsto a \in \text{GuardedAction} \Rightarrow$ 
    nextVar[ $\{\text{lhs}(a)\}$ ]  $\subseteq$  used( $g \mapsto a$ )
  @used_property1  $\forall g, a, st. st \in \text{State} \wedge g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \subseteq$ 
    dom( $st$ )  $\Rightarrow \text{semE}(\text{rhs}(a) \mapsto st) \neq \emptyset$ 
  @used_property2  $\forall g, a, st, st'. st \in \text{State} \wedge st' \in \text{State} \wedge g \mapsto a \in \text{GuardedAction} \wedge$ 
    used( $g \mapsto a$ )  $\triangleleft st = \text{used}(g \mapsto a) \triangleleft st' \Rightarrow \text{semE}(\text{rhs}(a) \mapsto st) = \text{semE}(\text{rhs}(a) \mapsto st')$ 
  @guardedActions_prop  $\forall g1, a1, g2, a2, st. g1 \mapsto a1 \in \text{GuardedAction} \wedge$ 
     $g2 \mapsto a2 \in \text{GuardedAction} \wedge st \in \text{State} \wedge (g1 \mapsto a1) \neq (g2 \mapsto a2) \wedge$ 
    used( $g1 \mapsto a1$ )  $\subseteq \text{dom}(st) \wedge \text{used}(g2 \mapsto a2) \subseteq \text{dom}(st)$ 
   $\wedge \text{semG}(g1 \mapsto (\text{ran}(\text{nextVar}) \triangleleft st)) = \text{TRUE} \wedge$ 
   $\text{semG}(g2 \mapsto (\text{ran}(\text{nextVar}) \triangleleft st)) = \text{TRUE} \Rightarrow \text{lhs}(a1) \neq \text{lhs}(a2)$ 
  @next_prop  $\forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \cap \text{ran}(\text{nextVar}) \neq \emptyset \Rightarrow$ 
     $\text{lhs}(a) \in \text{dom}(\text{nextVar})$ 
  @next_prop1  $\forall g, a, st. st \in \text{State} \wedge g \mapsto a \in \text{GuardedAction} \wedge$ 
     $\text{lhs}(a) \in \text{dom}(\text{nextVar}) \wedge \text{nextVar}(\text{lhs}(a)) \in \text{dom}(st)$ 
     $\Rightarrow \text{semE}(\text{rhs}(a) \mapsto st) = \{st(\text{nextVar}(\text{lhs}(a)))\}$ 
  @next_prop2  $\forall g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{lhs}(a) \in \text{dom}(\text{nextVar}) \Rightarrow$ 
     $\text{nextVar}(\text{lhs}(a)) \in \text{used}(g \mapsto a) \wedge (\text{ran}(\text{nextVar}) \setminus \{\text{nextVar}(\text{lhs}(a))\}) \cap \text{used}(g \mapsto a) = \emptyset$ 
  @reaction_noempty  $\exists g, a. g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \subseteq \text{ran}(\text{nextVar}) \wedge$ 
     $(\forall st. st \in \text{State} \Rightarrow \text{semG}(g \mapsto st) = \text{TRUE})$ 
  @enabled_ty  $\in \text{enabled} \subseteq (\text{GuardAction} \times \text{State})$ 
  @enabled_def  $\in \text{enabled} = \{g, a, st. g \mapsto a \in \text{GuardedAction} \wedge \text{used}(g \mapsto a) \subseteq \text{dom}(st)$ 
     $\wedge \text{semG}(g \mapsto \text{ran}(\text{nextVar}) \triangleleft st) = \text{TRUE} \mid (g \mapsto a \mapsto st)\}$ 
end

```

Listing B.1: Static features of the splitting refinement

Appendix C

mGAbs

```
machine mGAbs sees cGA

variables state

invariants
  @state_ty state ∈ State
  @init_invariant ran(nextVar) ⊆ dom(state)
events
  event INITIALISATION
  then
    @st state := init
  end
  event big_step
  any state1
  where
    @state_1 state1 ∈ State
    @state_2 state1 ≠ ∅

    @state_4 ∀ g, v, e. g ↦ (v ↦ e) ∈ GuardedAction ∧
      (g ↦ (v ↦ e)) ↦ state1 ∈ enabled ∧ v ∉ dom(nextVar) ⇒ v ∈ dom(state1) ∧
      state1(v) ∈ semE(e ↦ state1)
    @state_5 ∀ x . x ∈ dom(nextVar) ∩ dom(state1) ⇒ state1(x) = state(nextVar(x))
  then
    @state_re state := (ran(nextVar) ⊲ state) ⊲ state1
  end
end
```

Listing C.1: Static features of the splitting refinement

Appendix D

mGA

```

machine mGA refines mGAbs sees cGA

variables done r_state

invariants
  @r_state_ty r_state ∈ State
  @done_ty done ⊆ GuardedAction
  @done_used ∀ g, a. g ↦ a ∈ done ⇒ used( g ↦ a ) ⊆ dom(r_state)
  @dom_done_r_state dom(r_state) \ ran(nextVar) = prj1[prj2[done]] \ ran(nextVar)
  @done_semG ∀ g, a. g ↦ a ∈ done ⇒
    semG(g ↦ ran(nextVar) ≤ r_state) = TRUE
  @done_lhs ∀ g, a. g ↦ a ∈ done ⇒ lhs(a) ∈ dom(r_state)
  @done_r_state ∀ g, a. g ↦ a ∈ done ∧ lhs(a) ∉ dom(nextVar) ⇒
    r_state(lhs(a)) ∈ semE(rhs(a) ↦ r_state)
  @init_invariant ran(nextVar) ⊆ dom(r_state)
  @lhs_invariant ∀ g, a. g ↦ a ∈ GuardedAction \ done ∧ used(g ↦ a) ⊆ dom(r_state)
    ∧ semG(g ↦ ran(nextVar) ≤ r_state) = TRUE ⇒
    lhs(a) ∉ dom(r_state) \ ran(nextVar)
  @next_invariant ∀ g, a. g ↦ a ∈ done ∧ lhs(a) ∈ dom(nextVar) ∧ lhs(a) ∈ dom(r_state)
    ⇒ r_state(lhs(a)) = state(nextVar(lhs(a)))
  @next_done ∀ x. x ∈ ran(nextVar) ∧ r_state(x) ≠ state(x) ⇒ x ∈ prj1[prj2[done]]
  variant GuardedAction \ done

events
  event INITIALISATION
  then
    @rst r_state := init
    @init_done done := ∅
  end

  convergent event small_step
  any g a
  where
    @g_ty g ↦ a ∈ GuardedAction \ done
    @cl_ready used(g ↦ a) ⊆ dom(r_state)
    @guard_ok semG(g ↦ (ran(nextVar) ≤ r_state)) = TRUE
  then
    @a_cl r_state ≝ semA(a ↦ r_state)
    @d_cl done := done ∪ {g ↦ a}
  end

  event big_step refines big_step

  where
    @r_state_not_empty r_state ≠ ∅
    @finished ∀ g, a .
      (g ↦ a ∈ GuardedAction \ done ∧
       used(g ↦ a) ⊆ dom(r_state))
      ⇒ semG(g ↦ (ran(nextVar) ≤ r_state)) = FALSE

  with
    @state1 state1 = r_state

```

```
    then
      @pc done: =∅
      @rst r_state: =ran(nextVar) <= r_state
    end
  end
```

Listing D.1: Static features of the splitting refinement

Appendix E

cGACL

```

context cGACL extends cGA

sets Cluster

constants guardedAction clusterSize cl_used

axioms
  @clusterSize clusterSize ∈ Cluster → ℕ1
  @guardedAction_ty; guardedAction ∈ Cluster → (ℕ ⇔ GuardedAction)
  @guardedAction_dom ∀cl. (cl ∈ Cluster ⇒ dom(guardedAction(cl)) = 1..clusterSize(cl))
  @GuardAction_constrain GuardedAction = (UNION cl.
    cl ∈ Cluster | guardedAction(cl)[1..clusterSize(cl)] )
  @cl_used_ty cl_used ∈ Cluster → ℙ(Variable)
  @cl_used_def cl_used = (λcl. (cl ∈ Cluster) | used(guardedAction(cl)(1)))
  @size_spec ∀cl, i. i ∈ 1 .. clusterSize(cl) ⇒
    used(guardedAction(cl)(i)) ⊆ used(guardedAction(cl)(1)) ∪
    ({ g, a. g ↦ a ∈ guardedAction(cl)[1..i-1] | lhs(a) })

  @cluster_next ∀ cl, i. i ∈ 2..clusterSize(cl) ⇒ lhs(prj2(guardedAction(cl)
    (i-1))) ∈ used(guardedAction(cl)(i))
  @disjointClusters ∀cl1, cl2. ran(guardedAction(cl1)) ∩
    ran(guardedAction(cl2)) ≠ ∅ ⇒ cl1 = cl2
  @no_two_father ∀ cl, i, x. i ∈ 2 .. clusterSize(cl) ∧ x ∈ used(guardedAction(cl)(i))
    ⇒ (∀ ga1, ga2. ga1 ∈ GuardedAction ∧ ga2 ∈ GuardedAction ∧
      lhs(prj2(ga1)) = x ∧
      lhs(prj2(ga1)) = lhs(prj2(ga2)) ⇒ ga1 = ga2)
  @no_two_father2 ∀ cl, i. i ∈ 2 .. clusterSize(cl)
    ⇒ (∀ ga1, ga2. ga1 ∈ GuardedAction ∧ ga2 ∈ GuardedAction ∧
      lhs(prj2(ga1)) = lhs(prj2(guardedAction(cl)(i-1))) ∧
      lhs(prj2(ga1)) = lhs(prj2(ga2)) ⇒ ga1 = ga2)
  @used_cl_ga ∀ cl, i, j. i ∈ dom(guardedAction(cl)) ∧ j ∈ dom(guardedAction(cl)) ∧
    lhs(prj2(guardedAction(cl)(j))) ∈ used(guardedAction(cl)(i)) ∧
    lhs(prj2(guardedAction(cl)(i))) ∉ dom(nextVar) ⇒ (i > j)
  theorem @after_next ∀cl, i. i ∈ 2 .. clusterSize(cl) ⇒
    lhs(prj2(guardedAction(cl)(i-1))) ∉ ran(nextVar)
  theorem @intersection_empty ∀cl1, cl2, i, j. i ∈ dom(guardedAction(cl1)) ∧
    j ∈ dom(guardedAction(cl2)) ∧
    (cl1 ≠ cl2 ∨ i ≠ j) ⇒ guardedAction(cl1)(i) ≠ guardedAction(cl2)(j)
end

```

Listing E.1: Static features of the splitting refinement

Appendix F

mGACL

```

machine mGACL refines mGA sees cGACL

variables r_state pc

invariants
  @pc_ty pc ∈ Cluster → ℕ
  @pc_ok ∀cl · cl ∈ Cluster ⇒ pc(cl) ∈ 1 .. clusterSize(cl) + 1
  @done done = ( cl · cl ∈ Cluster | guardedAction(cl)[1.pc(cl)-1])
  @lhs_spec; ∀cl · cl ∈ Cluster ⇒ (lhs[prj2[guardedAction(cl)[1.pc(cl)-1]]] ⊆ dom(r_state))
  @r_state_invariant ∀cl · union(used[guardedAction(cl)[1.pc(cl)-1]]) ⊆ dom(r_state)
  @used_notin_r_state ∀cl, i · i > pc(cl) ∧ i ∈ dom(guardedAction(cl)) ⇒
    used(guardedAction(cl)(i)) ⊄ dom(r_state)

events
event INITIALISATION
  then
    @init_a r_state := init
    @todo_a pc := Cluster × {1}
  end

event small_step refines small_step
  any cl g a
  where
    @cl_ty cl ∈ Cluster
    @cl_ready cl_used(cl) ⊆ dom(r_state)
    @guard_act_dom pc(cl) ∈ dom(guardedAction(cl))
    @guard_act_val (g ↦ a) = guardedAction(cl)(pc(cl))
    @guard_ok semG(g ↦ ran(nextVar) ≤ r_state) = TRUE
  then
    @a_cl r_state ∈ semA(a ↦ r_state)
    @pc_next pc(cl) := pc(cl) + 1
  end

event big_step refines big_step
  where
    @r_state_not_empty r_state ≠ ∅
    @finished ∀cl · (cl ∈ Cluster ∧
      pc(cl) ∈ dom(guardedAction(cl)) ∧
      cl_used(cl) ⊆ dom(r_state))
      ⇒ semG(prj1(guardedAction(cl)(pc(cl))) ↦ ran(nextVar) ≤ r_state) = FALSE
  then
    @pc pc := Cluster × {1}
    @st r_state := ran(nextVar) ≤ r_state
  end
end

```

Listing F.1: Static features of the splitting refinement

Appendix G

The Class Diagram of Generated Java Code

ABSTRACT

In this report, we introduce a translation from SIGNAL into multi-threaded Java code and prove the semantics preservation of the translation based on Event-B. Intermediate representations: Synchronous Clock Guarded Action (S-CGA), Guarded Action (GA), and Guarded Action with CLusters (GACL) are used to divide the translation into the following 4 steps: (1) translate a SIGNAL program into an S-CGA, (2) translate the S-CGA into a GA, (3) translate the GA into a GACL, and (4) translate the GACL into multi-threaded Java code. Event-B is used to prove the semantics preservation of the translation from GA into GACL with its refinement framework. There are three levels for the refinement. Firstly, GA is specified by an Event-B model (denoted as *mGAbs*) with only a big step, which performs the computation of a reaction in an atomic step. Secondly, the model *mGAbs* is refined by decomposing the big step into a collection of small steps such that each small step corresponds to the computation of a single guarded action (the refined model denoted as *mGA*). Both *mGAbs* and *mGA* are models for GA. The model *mGA* is further refined by giving a partition to GA and each subset in the partition is called a cluster. The refined model is for GACL, denoted as *mGACL*. All proof obligations for these three levels of even-B models are proved to ensure the preservation of safety properties and deadlock freedom preservation.

KEYWORDS

Synchronous languages, SIGNAL, synchronous clock guarded actions, Guarded Action, Guarded Action with Clusters, Event-B, refinement, multi-thread Java code

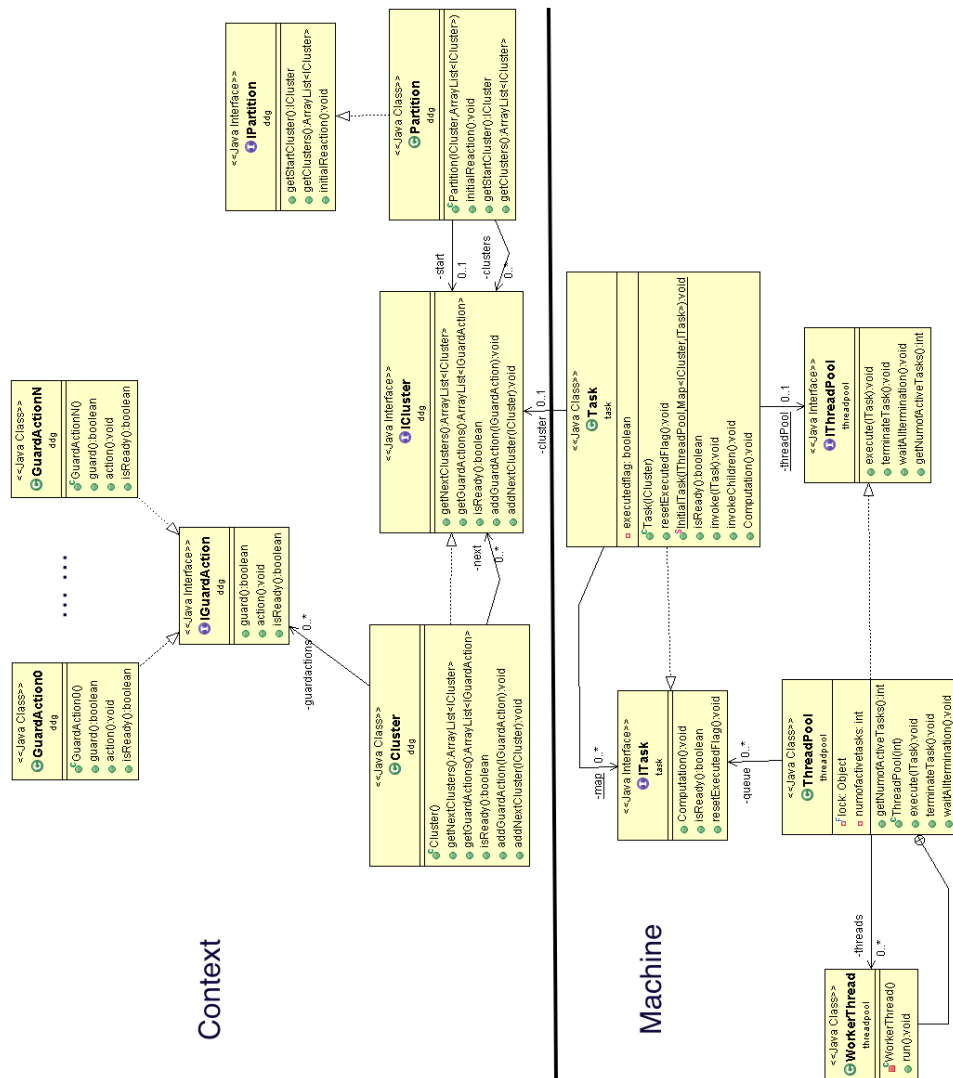


Figure G.1: The class diagram of the generated code